

Universidad Nacional de Colombia
Facultad de Ingeniería
Departamento de Ingeniería de Sistemas e Industrial



Proyecto Final - Sesión 3

UnTrade - Implementación, poblamiento y consultas

Ecosistema Transaccional Universitario

Integrantes del equipo:

José Carlos Robles Martínez (T.I. 1102838481)
Samuel Felipe Palacios Olaya (C.C. 1028480521)
Javier Miguel Vargas Méndez (C.C. 1067602299)
Miguel Silva Garzón (C.C. 1016949189)
Gabriel Esteban Medina Enciso (C.C. 1028861904)
Daniel Zambrano Ramírez (C.C. 1029144989)

Docente: Prof. Cristhian Fernando Lara Espejo
Bases de Datos I (2016353)

Julio de 2026

Índice general

1. Actualización del Esquema Relacional	4
1.1. Creación del Catálogo de Categorías	4
1.2. Flexibilización de Atributos de Publicación	5
2. Decisiones Arquitectónicas de Poblamiento	6
2.1. Motor Generador de Script DML (Python)	6
3. Scripts de Poblamiento de Datos	8
3.1. Tabla Principal y Volumen Transaccional	8
3.2. Código del Motor Generador de Datos (Python)	8
4. Lógica del Lado del Servidor (PL/SQL)	16
4.1. Triggers	16
4.1.1. Trigger Compra	16
4.1.2. Trigger trueque	16
4.1.3. Trigger prestamo	17
4.1.4. Trigger validación parar realizar compra	17
4.1.5. Evidencia de que los triggers funcionaron	18
4.2. Procedimientos	19
4.2.1. Aplicar Sanción	19
4.2.2. Calificar Vendedor	21
4.3. Funciones	22
4.3.1. Promedio	22
4.3.2. Probabilidad Producto	22
4.3.3. Cálculo de fiabilidad del vendedor	23
5. Batería de Consultas SQL	25
5.1. Nivel 1: Selección y Filtrado	25
5.1.1. Consulta N1-01	25
5.1.2. Consulta N1-02	25
5.1.3. Consulta N1-03	26
5.2. Nivel 2: JOINS Múltiples	27
5.2.1. Consulta N2-01	27
5.2.2. Consulta N2-02	29
5.2.3. Consulta N2-03	29
5.2.4. Consulta N2-04	30
5.2.5. Consulta N2-05	33
5.3. Nivel 3: Agregación	33
5.3.1. Consulta N3-01	33
5.3.2. Consulta N3-02	34
5.3.3. Consulta N3-03	36

5.3.4.	Consulta N3-04	36
5.4.	Nivel 4: Subconsultas	37
5.4.1.	Consulta N4-01	37
5.4.2.	Consulta N4-02	38
5.4.3.	Consulta N4-03	39
5.4.4.	Consulta N4-04	39
5.4.5.	Consulta N4-05	41
5.5.	Nivel 5: Funciones de Ventana	41
5.5.1.	Consulta N5-01	41
5.5.2.	Consulta N5-02	42
5.5.3.	Consulta N5-03	44
5.6.	Nivel 6: Vistas	44
5.6.1.	Consulta N6-01	44
5.6.2.	Consulta N6-02	47
5.6.3.	Consulta N6-03	49
5.6.4.	Consulta N6-04	49
5.6.5.	Consulta N6-05	50

Capítulo 1

Actualización del Esquema Relacional

Como parte del refinamiento y la iteración del diseño lógico presentado en sesiones previas, se identificaron mejoras críticas orientadas a incrementar la flexibilidad y robustez de nuestro modelo de datos, abordando particularmente la clasificación de los productos y la flexibilidad en la información de las publicaciones.

1.1. Creación del Catálogo de Categorías

En la versión anterior del esquema, el atributo referente a la categoría del producto se encontraba integrado directamente dentro de la tabla `PRODUCTO` (como un atributo `VARCHAR`). Esta estructura limitaba la gestión de categorías globales y presentaba riesgos de inconsistencia de datos debido a errores tipográficos.

Para solucionar este inconveniente, se implementó una tabla catálogo independiente denominada `CATEGORIA`, estructurada de la siguiente manera:

- `id_categoria`: Atributo entero y `AUTO_INCREMENT` que funciona como llave primaria.
- `nombre_categoria`: Atributo `VARCHAR(50)` que almacena el nombre oficial de la categoría.

Dado que un producto en el ecosistema universitario puede pertenecer simultáneamente a más de una categoría de búsqueda o área del conocimiento, se estableció una tabla asociativa denominada `CATEGORIA_PRODUCTO`. Dicha tabla intermedia aloja las llaves foráneas `id_categoria` e `id_producto`, siendo la llave primaria compuesta por la unión de ambas, lo que permite resolver correctamente la relación muchos a muchos (N:M).

Fragmento: Actualización Estructural DDL

```
1 CREATE TABLE CATEGORIA (  
2     id_categoria INT NOT NULL AUTO_INCREMENT,  
3     nombre VARCHAR(50) NOT NULL UNIQUE,  
4     PRIMARY KEY (id_categoria)  
5 );  
6  
7 CREATE TABLE CATEGORIA_PRODUCTO (  
8     id_categoria INT NOT NULL,  
9     id_producto INT NOT NULL,  
10    PRIMARY KEY (id_categoria, id_producto),  
11    CONSTRAINT fk_catprod_categoria FOREIGN KEY (id_categoria)  
12    REFERENCES CATEGORIA (id_categoria) ON DELETE CASCADE,  
13    CONSTRAINT fk_catprod_producto FOREIGN KEY (id_producto)  
14    REFERENCES PRODUCTO (id_producto) ON DELETE CASCADE  
15 );  
16  
17 -- Observación 1: La columna original "categoria" fue purgada de la tabla PRODUCTO.  
18 -- Observación 2: La limpieza inicial (DROP TABLES) respeta el nuevo orden inverso de dependencias.
```

1.2. Flexibilización de Atributos de Publicación

El segundo ajuste introducido en el DDL impacta a la entidad **PUBLICACION**. Atendiendo a casos de uso reales de la plataforma donde ciertos productos o servicios poseen un título lo suficientemente explícito como para prescindir de texto adicional, se determinó modificar la obligatoriedad de la descripción. Actualmente, el atributo **descripcion** permite valores **NULL**, ofreciendo a los vendedores la posibilidad de generar publicaciones de manera más ágil sin información redundante obligatoria.

Capítulo 2

Decisiones Arquitectónicas de Poblamiento

Para cumplir con los estrictos requisitos de volumen estipulados en la rúbrica (mínimo 1000 filas en tablas transaccionales y 10 filas en tablas maestras), el equipo de diseño optó por una arquitectura de **pre-generación procedural en memoria**.

En lugar de crear dependencias de ejecución en lenguajes externos durante el despliegue de la base de datos o realizar inserciones manuales propensas a errores, se desarrolló un motor generador. Este motor construye toda la lógica relacional en la memoria RAM y compila un único archivo SQL estático y ejecutable (`02_dml.sql`).

2.1. Motor Generador de Script DML (Python)

El motor fue desarrollado en Python (`script_de_población_de_datos.py`) empleando la librería **Faker** para construir un set de datos altamente realista y contextualizado. Es imperativo destacar que este script **no actúa como un cliente de base de datos** (no requiere conexión en tiempo de ejecución), sino como un compilador de texto avanzado que redacta sentencias SQL nativas.

Las decisiones clave que respaldan esta arquitectura son:

- **Eficiencia Computacional (Bulk Inserts):** Para procesar de manera eficiente las miles de filas transaccionales exigidas, el generador redacta sentencias `INSERT` multifila. Esto reduce drásticamente los tiempos de latencia y los costos de procesamiento en el motor de base de datos durante la evaluación, superando ampliamente el rendimiento de las sentencias individuales.
- **Integridad Referencial Absoluta en Memoria:** El script inicializa los catálogos fundacionales (Universidades, Materias, Categorías) mediante estructuras de datos nativas (arrays indexados y listas). Al generar las entidades dependientes (usuarios, publicaciones, compras), el motor cruza y asigna los identificadores directamente en memoria. Esto garantiza que el archivo SQL resultante sea consistente, es decir, que no viole las restricciones de integridad referencial por llaves foráneas.
- **Gestión Lógica de Restricciones (Constraints):**
 - Se aplicó un truncado de seguridad en la sintaxis (ej. `[:255]`) para respetar invariablemente los límites restrictivos de capacidad `VARCHAR` definidos en el DDL.
 - Se implementó una lógica matemática de saltos de tiempo (`timedelta`) para asegurar que los campos temporales (`fecha_solicitud`, `fecha_inicio`, `fecha_pactada`) de la tabla `PRESTAMO` superen estrictamente los chequeos de coherencia estipulados en las reglas `CHECK` del esquema.
- **Simulación de Triggers para Auditoría Inicial:** Dado que el despliegue del proyecto requiere ejecutar el poblamiento (`02_dml.sql`) antes de instanciar la lógica de lado del servidor (`03_plsql.sql`), los *triggers* de auditoría aún no existen en el motor durante la inserción masiva. Para solventar esto

y mantener la trazabilidad, el generador de Python fue programado para calcular e inyectar por sí mismo los registros en la tabla `AUDITORIA_TRANSACCIONES`, simulando el comportamiento exacto de los *triggers*.

En otras palabras, los triggers de auditoría deberían ser creados previo a la inserción masiva de datos, pero para mantener el flujo especificado el documento decidimos agregar manualmente las inserciones que normalmente serían responsabilidad del trigger (aprovechando la simplicidad de las inserciones iniciales para las tablas auditadas), de modo que toda la base de datos, una vez completado el flujo de archivos `.sql`, quedará consistente y seguirá la estructura deseada dadas las reglas del negocio que hemos establecido hasta ahora.

Capítulo 3

Scripts de Poblamiento de Datos

En este capítulo presentamos el código fuente íntegro del motor generador que produce el entregable analítico final.

3.1. Tabla Principal y Volumen Transaccional

Escogimos como entidad principal la tabla PUBLICACIÓN en calidad de su definición como el elemento por medio del cual se rigen las interacciones principales entre usuarios (el objetivo de los vendedores es crear publicaciones atractivas a los clientes, y la meta principal de los compradores es buscar la publicación que más se ajusta a sus necesidades), así como el hecho de que es la entidad más central en el modelo relacional (está directamente conectada a casi todas las entidades del negocio).

Las iteraciones del ciclo principal -del algoritmo en python- de publicaciones se proyectaron a un volumen estricto de 1000 registros, dando pie a 1050 transacciones en la tabla de COMPRA. Al gozar de un catálogo robusto, las ventas se distribuyen estadísticamente, garantizando resultados contundentes y representativos en las funciones de agregación (GROUP BY, SUM) y análisis por ventanas de la batería de consultas posterior.

3.2. Código del Motor Generador de Datos (Python)

El siguiente script compila en memoria las reglas de negocio, resuelve las jerarquías asociativas y exporta el archivo final 02_dml.sql con sintaxis INSERT multifila nativa para el sistema gestor (02_dml.sql será adjuntado junto con los demás entregables).

script_de_poblacion_de_datos.py

```
1 import random
2 from faker import Faker
3 from datetime import timedelta
4
5 # Inicializar Faker con localizacion colombiana
6 fake = Faker('es_CO')
7
8 universidades_data = [
9     ('Universidad_Nacional_de_Colombia', 'Colombia', '@unal.edu.co'),
10    ('Universidad_de_los_Andes', 'Colombia', '@uniandes.edu.co'),
11    ('Universidad_Javeriana', 'Colombia', '@javeriana.edu.co'),
12    ('Universidad_de_Antioquia', 'Colombia', '@udea.edu.co'),
13    ('Universidad_del_Valle', 'Colombia', '@univalle.edu.co'),
14    ('Universidad_Industrial_de_Santander', 'Colombia', '@uis.edu.co'),
15    ('Universidad_del_Rosario', 'Colombia', '@urosario.edu.co'),
16    ('Universidad_de_la_Sabana', 'Colombia', '@unisabana.edu.co'),
17    ('Universidad_EAFIT', 'Colombia', '@eafit.edu.co'),
```



```

18     ('Universidad_del_Norte', 'Colombia', '@uninorte.edu.co')
19 ]
20
21 materias_data = [
22     (1, 'Bases_de_Datos_I', 3),
23     (1, 'Programacion_Orientada_a_Objeto', 4),
24     (1, 'Calculo_Diferencial', 4),
25     (1, 'Fisica_Mecanica', 4),
26     (1, 'Estructuras_de_Datos', 3),
27     (2, 'Arquitectura_de_Software', 3),
28     (2, 'Ingenieria_de_Requisitos', 3),
29     (3, 'Redes_de_Computadores', 4),
30     (4, 'Sistemas_Operativos', 4),
31     (5, 'Inteligencia_Artificial', 3)
32 ]
33
34 categorias_data = [
35     'Libros_y_Textos', 'Calculadoras_y_Computo', 'Material_de_Laboratorio',
36     'Electronica_y_Robotica', 'Arquitectura_y_Diseño', 'Medicina_y_Salud',
37     'Papeleria_Tecnica', 'Software_y_Licencias', 'Instrumentos_de_Medicion',
38     'Herramientas_de_Taller'
39 ]
40
41 # -----
42 # FUNCIONES AUXILIARES
43 # -----
44 def format_val(val):
45     """Formatea valores de Python para sintaxis SQL."""
46     if val is None:
47         return "NULL"
48     if isinstance(val, (int, float)):
49         return str(val)
50     return f"'{str(val).replace(chr(39), chr(39)+chr(39))}'"
51
52 def write_bulk_insert(file, table, columns, values_list):
53     """Genera y escribe una sentencia INSERT multifila en el archivo."""
54     if not values_list:
55         return
56
57     cols_str = ",".join(columns)
58     file.write(f"INSERT INTO {table} ({cols_str}) VALUES\n")
59
60     rows = []
61     for vals in values_list:
62         vals_str = ",".join(format_val(v) for v in vals)
63         rows.append(f"({vals_str})")
64
65     file.write(",\n".join(rows) + ";\n\n")
66
67 # -----
68 # GENERACION DE DATOS (EN MEMORIA)
69 # -----
70 print("Generando datos en memoria...")
71
72 # Universidades, Materias, Categorías
73 univ_vals = []
74 universities_ref = []
75 for i, (nombre, pais, dominio) in enumerate(universidades_data, start=1):
76     univ_vals.append((nombre, pais, dominio))

```

```

77 universities_ref.append((i, dominio))
78
79 materia_vals = materias_data.copy()
80 materia_ids = list(range(1, len(materias_data) + 1))
81
82 categoria_vals = [(cat,) for cat in categorias_data]
83 categoria_ids = list(range(1, len(categorias_data) + 1))
84
85 # Usuarios, Admin, Vendedor, Comprador
86 usuario_vals = []
87 user_ids = []
88 for i in range(1, 301):
89     univ_id, domain = random.choice(universities_ref)
90     base_name = fake.unique.user_name()[:25]
91     email = f"{base_name}{random.randint(100,999)}{domain}"
92     usuario_vals.append((univ_id, fake.name()[:50], email, fake.sha256()))
93     user_ids.append(i)
94
95 admin_vals = []
96 areas_soporte_realistas = [
97     'Soporte_Tecnico_de_Plataforma', 'Moderacion_de_Contenido', 'Seguridad_y
98     Cuentas',
99     'Gestion_de_Prestamos_y_Multas', 'Atencion_al_Estudiente', 'Verificacion_de
100     Identidad',
101     'Resolucion_de_Disputas', 'Soporte_de_Pagos', 'Auditoria_de_Transacciones', '
102     Administracion_General'
103 ]
104 admin_ids = user_ids[:10]
105 for i in range(10):
106     admin_vals.append((admin_ids[i], random.choice(['SuperAdmin', 'Moderador', '
107     Soporte']), areas_soporte_realistas[i]))
108
109 vendedor_vals = []
110 vendedor_ids = random.sample(user_ids, 150)
111 for u_id in vendedor_ids:
112     calificacion = random.choice([0.0, 10.0, round(random.uniform(3.5, 5.0), 1)])
113     ventas = random.randint(0, 50)
114     vendedor_vals.append((u_id, calificacion, ventas))
115
116 comprador_vals = []
117 preferencias_academicas = [
118     'Libros_de_calculo_y_matematicas', 'Calculadoras_cientificas_graficas', '
119     Implementos_de_laboratorio_de_quimica',
120     'Materiales_de_arquitectura_y_dibujo', 'Tarjetas_de_desarrollo_Arduino_y
121     sensores', 'Apuntes_y_cuadernos_de_ingenieria',
122     'Libros_de_fisica_mecanica', 'Kits_de_electronica_basica', 'Libros_de
123     programacion_y_bases_de_datos', 'Estetoscopios_y_material_medico'
124 ]
125 comprador_ids = random.sample(user_ids, 250)
126 for u_id in comprador_ids:
127     preferencia = random.choice(preferencias_academicas)
128     comprador_vals.append((u_id, preferencia[:255]))
129
130 banco_productos_academicos = [
131     ("Libro_de_Calculo_de_Larson_11Ed", "Libro_en_excelente_estado,_pasta_original.
132     ", 95000),
133     ("Calculadora_Cientifica_Casio", "Calculadora_con_funciones_matriciales,_poco
134     uso.", 75000),

```

```

126     ("Kit_de_Laboratorio_de_Quimica", "Set_completo_de_cristaleria_resistente_al_
127     calor.", 60000),
128     ("Multimetro_Digital_UT33C+", "Medicion_de_temperatura_y_continuidad.", 45000),
129     ("Escalimetro_profesional", "Implementos_de_dibujo_tecnico_seminuevos.", 50000)
130 ],
131     ("Tarjeta_Arduino_Uno_R3", "Kit_de_desarrollo_para_sistemas_embebidos.", 90000)
132 ],
133     ("Libro_de_Bases_de_Datos", "Edicion_en_espanol,_clave_para_ingenieria.",
134     110000),
135     ("Estetoscopio_Littmann_Classic_III", "Uso_exclusivo_de_area_de_salud.",
136     350000),
137     ("Prototyping_Breadboard", "Placa_de_pruebas_protoboard_con_juego_de_cables.",
138     25000),
139     ("Fisica_Universitaria_Sears", "Conjunto_de_ambos_volumenes_empastados.",
140     140000)
141 ]
142
143 publicacion_vals = []
144 producto_vals = []
145 servicio_vals = []
146 cat_prod_vals = []
147 publicacion_ids = []
148 producto_ids = []
149
150 pub_id_counter = 0
151 prod_id_counter = 0
152 cat_prod_set = set()
153
154 for _ in range(1000):
155     v_id = random.choice(vendedor_ids)
156     a_id = random.choice(admin_ids) if random.random() > 0.2 else None
157     tipo = random.choice(['Producto', 'Servicio'])
158     estado = random.choice(['Activa', 'Pausada', 'Bloqueada', 'Finalizada'])
159
160     pub_id_counter += 1
161     publicacion_ids.append(pub_id_counter)
162
163     if tipo == 'Producto':
164         prod_base = random.choice(banco_productos_academicos)
165         titulo = prod_base[0] + f"_(Ref:_{random.randint(10,99)})"
166         descripcion = prod_base[1] if random.random() > 0.3 else None
167         precio = float(prod_base[2] + random.randint(-5000, 10000))
168
169         publicacion_vals.append((v_id, a_id, tipo, titulo, descripcion, estado))
170
171         prod_id_counter += 1
172         producto_ids.append(prod_id_counter)
173         stock = random.choice([0, random.randint(1, 10)])
174         producto_vals.append((pub_id_counter, precio, round(random.uniform(3.0,
175         10.0), 1), random.choice(['NUEVO', 'USADO']), stock))
176
177     categorias_asignadas = random.sample(categoria_ids, random.randint(1, 2))
178     for cat_id in categorias_asignadas:
179         if (cat_id, prod_id_counter) not in cat_prod_set:
180             cat_prod_vals.append((cat_id, prod_id_counter))
181             cat_prod_set.add((cat_id, prod_id_counter))
182
183     else:
184
185 servicios_academicos = [

```

```

177     ("Tutoria_de_Calculo_Diferencial", "Clases_orientadas_a_parciales.",
178         35000),
179     ("Asesoria_en_Programacion", "Apoyo_en_codigo_C++_y_Java.", 45000),
180     ("Clases_de_Fisica_Mecanica", "Diagramas_de_cuerpo_libre.", 40000),
181     ("Correccion_de_Articulos", "Revision_de_estilo_en_ingles.", 50000)
182 ]
183 serv_base = random.choice(servicios_academicos)
184 descripcion = serv_base[1] if random.random() > 0.3 else None
185 publicacion_vals.append((v_id, a_id, tipo, serv_base[0], descripcion,
186     estado))
187 servicio_vals.append((pub_id_counter, random.choice(['Presencial', 'Virtual
188     ']), float(serv_base[2]), "L-V 2pm a 6pm", round(random.uniform(4.0,
189     10.0), 1)))
190
191 # Auditoria
192 auditoria_vals = []
193 usuario_auditor_script = "root@localhost"
194
195 # Materia-Producto
196 mat_prod_vals = []
197 mat_prod_set = set()
198 for _ in range(60):
199     m_id = random.choice(materia_ids)
200     p_id = random.choice(producto_ids)
201     if (m_id, p_id) not in mat_prod_set:
202         mat_prod_vals.append((m_id, p_id))
203         mat_prod_set.add((m_id, p_id))
204
205 # Compras y Simulacion del Trigger de Compras
206 compra_vals = []
207 for i in range(1050):
208     compra_id = i + 1
209     c_id = random.choice(comprador_ids)
210     pub_id = random.choice(publicacion_ids)
211     monto = round(random.uniform(20000.0, 350000.0), 2)
212     metodo_pago = random.choice(['Transferencia_Bancaria', 'Nequi', 'Daviplata', '
213     Efectivo'])
214     compra_vals.append((c_id, pub_id, monto, metodo_pago))
215
216 # Registro de Auditoria (Simulando trg_auditar_nueva_compra)
217 detalle_compra = f"El comprador ID {c_id} realizo una compra en la publicacion
218     ID {pub_id} por un monto de ${monto} con el metodo de pago: {metodo_pago}"
219 auditoria_vals.append((compra_id, None, None, 'NUEVA_COMPRA', detalle_compra,
220     usuario_auditor_script))
221
222 # Ofertas
223 oferta_vals = []
224 for _ in range(200):
225     c_id = random.choice(comprador_ids)
226     pub_id = random.choice(publicacion_ids)
227     monto_ofertado = round(random.uniform(15000.0, 120000.0), 2)
228     estado_oferta = random.choice(['Pendiente', 'Aceptada', 'Rechazada'])
229     oferta_vals.append((c_id, pub_id, monto_ofertado, estado_oferta))
230
231 # Prestamos y Simulacion del Trigger de Prestamos
232 prestamo_vals = []
233 prestamos_demorados = []

```

```

2229 prestamo_id_counter = 0
2230
2231 for _ in range(200):
2232     prestamo_id_counter += 1
2233     c_id = random.choice(comprador_ids)
2234     pub_id = random.choice(publicacion_ids)
2235     f_solicitud = fake.date_time_between(start_date='-6m', end_date='-1m')
2236     f_inicio = f_solicitud + timedelta(days=1)
2237     f_pactada = f_inicio + timedelta(days=random.randint(5, 15))
2238
2239     estado_prestamo = random.choice(['Solicitado', 'Activo', 'Devuelto', 'Demorado'])
2240     f_real = None
2241     if estado_prestamo == 'Devuelto':
2242         f_real = f_pactada - timedelta(days=random.randint(0, 2))
2243
2244     if estado_prestamo == 'Demorado':
2245         prestamos_demorados.append((prestamo_id_counter, c_id))
2246
2247     prestamo_vals.append((c_id, pub_id, f_solicitud.strftime('%Y-%m-%d_%H:%M:%S'),
2248                          f_inicio.strftime('%Y-%m-%d_%H:%M:%S'),
2249                          f_pactada.strftime('%Y-%m-%d_%H:%M:%S'), f_real.strftime(
2250                              '%Y-%m-%d_%H:%M:%S') if f_real else None,
2251                          estado_prestamo))
2252
2253     # Registro de Auditoria (Simulando trg_auditar_nuevo_prestamo)
2254     detalle_prestamo = f"El comprador ID {c_id} solicitó en préstamo la publicación ID {pub_id}. Fecha pactada de devolución: {f_pactada.strftime('%Y-%m-%d_%H:%M:%S')}."
2255     auditoria_vals.append((None, None, prestamo_id_counter, 'NUEVO_PRESTAMO',
2256                          detalle_prestamo, usuario_auditor_script))
2257
2258     # Sanciones
2259     sancion_vals = []
2260     for p_id, c_id in prestamos_demorados:
2261         admin_id = random.choice(admin_ids)
2262         motivo_sancion = "Incumplimiento en la devolución del material."
2263         monto_multa = round(random.uniform(15000.0, 50000.0), 2)
2264         f_inicio_sancion = fake.date_time_between(start_date='-20d', end_date='now')
2265         f_fin_sancion = f_inicio_sancion + timedelta(days=30)
2266
2267         sancion_vals.append((c_id, p_id, admin_id, motivo_sancion, monto_multa,
2268                             f_inicio_sancion.strftime('%Y-%m-%d_%H:%M:%S'), f_fin_sancion.strftime('%Y-%m-%d_%H:%M:%S'), 'Vigente'))
2269
2270     # Trueques y Simulación del Trigger de Trueques
2271     trueque_vals = []
2272     for i in range(80):
2273         trueque_id = i + 1
2274         c_id = random.choice(comprador_ids)
2275         pub_ofrecida, pub_deseada = random.sample(publicacion_ids, 2)
2276         estado_trueque = random.choice(['Pendiente', 'Aceptado', 'Rechazado'])
2277         trueque_vals.append((c_id, pub_deseada, pub_ofrecida, estado_trueque))
2278
2279     # Registro de Auditoria (Simulando trg_auditar_nuevo_trueque)
2280     detalle_trueque = f"Trueque iniciado por comprador ID {c_id} ofreciendo publicación {pub_ofrecida} por la publicación {pub_deseada}."
2281     auditoria_vals.append((None, trueque_id, None, 'NUEVO_TRUEQUE', detalle_trueque,
2282                          usuario_auditor_script))

```

```

277 -----
278 # -----
279 # GENERACION DEL SCRIPT SQL (ESCRITURA A ARCHIVO)
280 # -----
281 print("Escribiendo datos en 02_dml.sql...")
282
283 with open('02_dml.sql', 'w', encoding='utf-8') as sql_file:
284     sql_file.write("--=====\\n")
285     sql_file.write("--Script de Poblacion de Datos (DML)\\n")
286     sql_file.write("--Generado automaticamente (Inserts Multifila y simulacion de triggers)\\n")
287     sql_file.write("--=====\\n\\n")
288
289     sql_file.write("USE UnTrade;\\n\\n")
290
291     sql_file.write("--1. Maestros (Universidades, Materias, Categorias)\\n")
292     write_bulk_insert(sql_file, "UNIVERSIDAD", ["nombre", "pais", "dominio_correo"], univ_vals)
293     write_bulk_insert(sql_file, "MATERIA", ["id_universidad", "nombre_materia", "creditos"], materia_vals)
294     write_bulk_insert(sql_file, "CATEGORIA", ["nombre"], categoria_vals)
295
296     sql_file.write("--2. Usuarios, Administradores, Vendedores, Compradores\\n")
297     write_bulk_insert(sql_file, "USUARIO", ["id_universidad", "nombre_completo", "correo_estudiantil", "password_hash"], usuario_vals)
298     write_bulk_insert(sql_file, "ADMINISTRADOR", ["id_administrador", "nivel_permiso", "area_soporte"], admin_vals)
299     write_bulk_insert(sql_file, "VENDEDOR", ["id_vendedor", "calificacion", "ventas_completadas"], vendedor_vals)
300     write_bulk_insert(sql_file, "COMPRADOR", ["id_comprador", "preferencias_busqueda"], comprador_vals)
301
302     sql_file.write("--3. Publicaciones, Productos, Servicios y Categorias Asociadas\\n")
303     write_bulk_insert(sql_file, "PUBLICACION", ["id_vendedor", "id_administrador_moderador", "tipo_item", "titulo", "descripcion", "estado_publicacion"], publicacion_vals)
304     write_bulk_insert(sql_file, "PRODUCTO", ["id_publicacion", "precio", "calificacion", "estado_fisico", "stock"], producto_vals)
305     write_bulk_insert(sql_file, "CATEGORIA_PRODUCTO", ["id_categoria", "id_producto"], cat_prod_vals)
306     write_bulk_insert(sql_file, "SERVICIO", ["id_publicacion", "modalidad", "tarifa_por_hora", "disponibilidad_horaria", "calificacion"], servicio_vals)
307
308     sql_file.write("--4. Transacciones y Operaciones (Compras, Ofertas, Trueques, Prestamos)\\n")
309     write_bulk_insert(sql_file, "MATERIA_PRODUCTO", ["id_materia", "id_producto"], mat_prod_vals)
310     write_bulk_insert(sql_file, "COMPRA", ["id_comprador", "id_publicacion", "monto_total", "metodo_pago"], compra_vals)
311     write_bulk_insert(sql_file, "OFERTA", ["id_comprador", "id_publicacion", "monto_ofertado", "estado_oferta"], oferta_vals)
312     write_bulk_insert(sql_file, "PRESTAMO", ["id_comprador", "id_publicacion", "fecha_solicitud", "fecha_inicio", "fecha_devolucion_pactada", "fecha_devolucion_real", "estado_prestamo"], prestamo_vals)
313     write_bulk_insert(sql_file, "SANCION", ["id_usuario", "id_prestamo", "id_administrador", "motivo", "monto_multa", "fecha_inicio", "fecha_fin", "estado_sancion"], sancion_vals)

```

```
314 write_bulk_insert(sql_file, "TRUEQUE", ["id_comprador_iniciador", "  
315 id_publicacion_deseada", "id_publicacion_ofrecida", "estado_trueque"],  
316 trueque_vals)  
317  
318 sql_file.write("--5. Tablas de Log y Auditoria (Simulacion de Triggers)\n")  
319 write_bulk_insert(sql_file, "AUDITORIA_TRANSACCIONES", ["id_compra", "  
id_trueque", "id_prestamo", "tipo_evento", "detalle_evento", "  
usuario_auditor"], auditoria_vals)  
320  
321 print("Script finalizado exitosamente. Auditoria generada correctamente.")
```

Capítulo 4

Lógica del Lado del Servidor (PL/SQL)

4.1. Triggers

Los triggers implementados en el esquema de base de datos tienen el propósito de automatizar las reglas de negocio, garantizar la integridad referencial antes de las transacciones y registrar auditorías de eventos críticos de manera transparente, asegurando que las validaciones y el historial operen independientemente de la capa de aplicación.

4.1.1 Trigger Compra

Descripción: Este disparador se activa automáticamente después de cada inserción exitosa en la tabla de compras, capturando los datos clave de la transacción como el identificador del comprador, la publicación adquirida, el monto, el método de pago y el usuario del sistema que ejecuta la acción.

Utilidad para el modelo: Permite mantener un registro histórico inmutable en la tabla de auditoría, lo cual es esencial para rastrear los movimientos financieros, resolver posibles disputas comerciales y monitorear la actividad transaccional del sistema sin requerir lógica adicional de seguimiento en el frontend.

```
1 CREATE TRIGGER trg_auditar_nueva_compra
2 AFTER INSERT ON COMPRA
3 FOR EACH ROW
4 BEGIN
5     INSERT INTO AUDITORIA_TRANSACCIONES (id_compra, tipo_evento, detalle_evento, usuario_auditor)
6     VALUES
7     (NEW.id_compra, 'NUEVA_COMPRA', CONCAT('El comprador ID ', NEW.id_comprador, ' realizó una
8     compra en la publicación ID ', NEW.id_publicacion, ' por un monto de $', NEW.monto_total, ' con
9     el método de pago: ', NEW.metodo_pago), CURRENT_USER());
10 END //
```

4.1.2 Trigger trueque

Descripción: Este trigger registra un evento en la tabla de auditoría inmediatamente después de que se inserta un nuevo registro de trueque en la base de datos, documentando quién inició el proceso y qué publicaciones exactas están involucradas en el intercambio.

Utilidad para el modelo: Es fundamental para el control operativo de los trueques, proporcionando trazabilidad directa sobre los intercambios de artículos entre los usuarios, facilitando así la labor de los administradores ante posibles reclamaciones o fraudes en la entrega de bienes.

```
1 CREATE TRIGGER trg_auditar_nuevo_trueque
2 AFTER INSERT ON TRUEQUE
3 FOR EACH ROW
4 BEGIN
```



```

5 INSERT INTO AUDITORIA_TRANSACCIONES (id_trueque, tipo_evento, detalle_evento, usuario_auditor)
6 VALUES
7 (NEW.id_trueque, 'NUEVO_TRUEQUE', CONCAT('Trueque iniciado por comprador ID ', NEW.
  id_comprador_iniciador, ' ofreciendo publicación ', NEW.id_publicacion_ofrecida, ' por la
  publicación ', NEW.id_publicacion_deseada), CURRENT_USER());
8 END//

```

4.1.3 Trigger prestamo

Descripción: Disparador que se ejecuta tras la inserción de una nueva solicitud de préstamo, registrando de forma automática en la tabla de auditoría el identificador del solicitante, el artículo prestado y la fecha exacta pactada para su devolución.

Utilidad para el modelo: Otorga visibilidad y control temporal sobre las operaciones de préstamo o alquiler de ítems, asegurando que quede constancia inalterable del compromiso adquirido por el usuario solicitante para el monitoreo de plazos vencidos.

```

1 CREATE TRIGGER trg_auditar_nuevo_prestamo
2 AFTER INSERT ON PRESTAMO
3 FOR EACH ROW
4 BEGIN
5 INSERT INTO AUDITORIA_TRANSACCIONES (id_prestamo, tipo_evento, detalle_evento, usuario_auditor)
6 VALUES
7 (NEW.id_prestamo, 'NUEVO_PRESTAMO', CONCAT('El comprador ID ', NEW.id_comprador, ' solicitó
  en préstamo la publicación ID ', NEW.id_publicacion, '. Fecha pactada de devolución: ',
  DATE_FORMAT(NEW.fecha_devolucion_pactada, '%Y-%m-%d %H:%i'), CURRENT_USER());
8 END //

```

4.1.4 Trigger validación parar realizar compra

Descripción: Este trigger preventivo se ejecuta antes de insertar una compra. Revisa la existencia tanto del comprador como de la publicación. Posteriormente, valida las existencias lógicas: si el ítem es un "Producto", exige que su stock sea mayor a cero; si es un "Servicio", comprueba que posea disponibilidad horaria registrada.

Utilidad para el modelo: Actúa como una barrera estricta de validación y consistencia de datos que previene la compra de artículos agotados o servicios sin horario, protegiendo la integridad transaccional del mercado e impidiendo fallos lógicos graves que afectarían la experiencia del usuario.

```

1 CREATE TRIGGER trg_validar_disponibilidad_compra
2 BEFORE INSERT ON COMPRA
3 FOR EACH ROW
4 BEGIN
5
6 DECLARE v_existe_comprador INT DEFAULT 0;
7 DECLARE v_existe_publicacion INT DEFAULT 0;
8 DECLARE v_tipo_item VARCHAR(20);
9 DECLARE v_stock_producto INT;
10 DECLARE v_disp_servicio VARCHAR(255);
11
12 SELECT COUNT(*) INTO v_existe_comprador
13 FROM COMPRADOR WHERE id_comprador = NEW.id_comprador;
14
15 IF v_existe_comprador = 0 THEN
16 SIGNAL SQLSTATE '45000'
17 SET MESSAGE_TEXT = 'Excepción: El comprador indicado no existe en el sistema.';
18 END IF;
19
20 SELECT COUNT(*) INTO v_existe_publicacion
21 FROM PUBLICACION WHERE id_publicacion = NEW.id_publicacion;

```

```

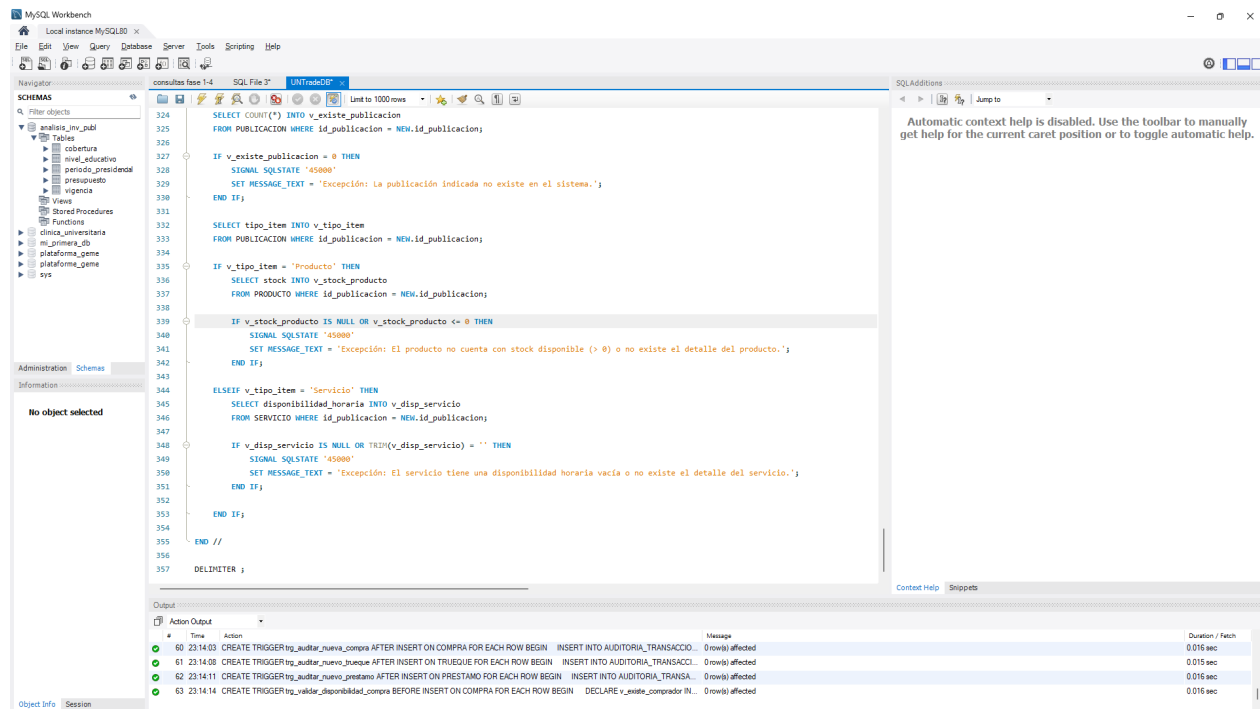
22 -----
23 ----- IF v_existe_publicacion = 0 THEN
24 -----     SIGNAL SQLSTATE '45000'
25 -----     SET MESSAGE_TEXT = 'Excepción: La publicación indicada no existe en el sistema.';
26 ----- END IF;
27 -----
28 ----- SELECT tipo_item INTO v_tipo_item
29 ----- FROM PUBLICACION WHERE id_publicacion = NEW.id_publicacion;
30 -----
31 ----- IF v_tipo_item = 'Producto' THEN
32 -----     SELECT stock INTO v_stock_producto
33 -----     FROM PRODUCTO WHERE id_publicacion = NEW.id_publicacion;
34 -----
35 ----- IF v_stock_producto IS NULL OR v_stock_producto <= 0 THEN
36 -----     SIGNAL SQLSTATE '45000'
37 -----     SET MESSAGE_TEXT = 'Excepción: El producto no cuenta con stock disponible (> 0) o no
38 -----     existe el detalle del producto.';
39 ----- END IF;
40 -----
41 ----- ELSEIF v_tipo_item = 'Servicio' THEN
42 -----     SELECT disponibilidad_horaria INTO v_disp_servicio
43 -----     FROM SERVICIO WHERE id_publicacion = NEW.id_publicacion;
44 -----
45 ----- IF v_disp_servicio IS NULL OR TRIM(v_disp_servicio) = '' THEN
46 -----     SIGNAL SQLSTATE '45000'
47 -----     SET MESSAGE_TEXT = 'Excepción: El servicio tiene una disponibilidad horaria vacía o no
48 -----     existe el detalle del servicio.';
49 ----- END IF;
50 -----
51 ----- END //

```

4.1.5 Evidencia de que los triggers funcionaron

Descripción: Esta sección contiene las representaciones gráficas de los resultados obtenidos tras la ejecución de los disparadores dentro del entorno de base de datos, validando visualmente que las reglas de negocio y restricciones operan según lo programado.

Utilidad para el modelo: Brinda soporte documental esencial sobre las pruebas de integración del sistema, garantizando que el comportamiento dinámico de la base de datos reacciona adecuadamente ante los estímulos de transacciones y cumple con la robustez requerida.



4.2. Procedimientos

Los procedimientos almacenados se diseñaron para encapsular operaciones complejas y transaccionales que involucran múltiples tablas, ofreciendo tanto a los administradores como a las interfaces de la aplicación métodos seguros, optimizados y estandarizados para modificar el estado del sistema sin exponer la lógica interna de manipulación.

4.2.1 Aplicar Sanción

Descripción: Este procedimiento gestiona la imposición de una sanción económica. Primero valida la existencia del administrador y el préstamo; luego verifica que la fecha de devolución haya vencido. De ser así, inserta la penalización correspondiente y actualiza el estado del préstamo a "Demorado", agrupando toda la operación en una sola transacción.

Utilidad para el modelo: Centraliza la lógica de penalizaciones, previniendo inconsistencias al asegurar que las multas solo se apliquen bajo condiciones válidas. Además, automatiza el cambio de estado de la operación original, simplificando el mantenimiento de las políticas punitivas del modelo.

```

1 CREATE PROCEDURE aplicar_sancion(
2     IN p_id_prestamo INT,
3     IN p_id_administrador INT,
4     IN p_motivo VARCHAR(255),
5     IN p_monto_incremento DECIMAL(10,2)
6 )
7 BEGIN
8     DECLARE v_existe_prestamo INT DEFAULT 0;
9     DECLARE v_existe_administrador INT DEFAULT 0;
10    DECLARE v_id_usuario INT;
11    DECLARE v_fecha_pago_oportuno DATETIME;
12    DECLARE v_fecha_devolucion_real DATETIME;
13    DECLARE v_nuevo_monto DECIMAL(10,2);
14    START TRANSACTION;

```

```

15 SELECT COUNT(*)
16 INTO v_existe_administrador
17 FROM ADMINISTRADOR
18 WHERE id_administrador = p_id_administrador;
19
20 IF v_existe_administrador = 0 THEN
21     SIGNAL SQLSTATE '45000'
22     SET MESSAGE_TEXT = 'El administrador no existe.';
23 END IF;
24 SELECT COUNT(*)
25 INTO v_existe_prestamo
26 FROM PRESTAMO
27 WHERE id_prestamo = p_id_prestamo;
28
29 IF v_existe_prestamo = 0 THEN
30     SIGNAL SQLSTATE '45000'
31     SET MESSAGE_TEXT = 'El préstamo no existe.';
32 END IF;
33 SELECT
34     id_comprador,
35     fecha_devolucion_pactada,
36     fecha_devolucion_real
37 INTO
38     v_id_usuario,
39     v_fecha_pago_oportuno,
40     v_fecha_devolucion_real
41 FROM PRESTAMO
42 WHERE id_prestamo = p_id_prestamo;
43 IF NOW() <= v_fecha_pago_oportuno THEN
44     SIGNAL SQLSTATE '45000'
45     SET MESSAGE_TEXT =
46         'La sanción no es válida: todavía no ha vencido la fecha de devolución pactada.';
47 END IF;
48 SET v_nuevo_monto = p_monto_incremento;
49 INSERT INTO SANCION (
50     id_usuario,
51     id_prestamo,
52     id_administrador,
53     motivo,
54     monto_multa,
55     fecha_inicio,
56     fecha_fin,
57     estado_sancion
58 )
59 VALUES (
60     v_id_usuario,
61     p_id_prestamo,
62     p_id_administrador,
63     p_motivo,
64     v_nuevo_monto,
65     NOW(),
66     NULL,
67     'Vigente'
68 );
69 UPDATE PRESTAMO
70 SET estado_prestamo = 'Demorado'
71 WHERE id_prestamo = p_id_prestamo;
72
73 COMMIT;

```

74 END

4.2.2 Calificar Vendedor

Descripción: Procedimiento que hace uso de un cursor estructurado para iterar sobre todas las calificaciones de los productos asociados a un vendedor en específico. Efectúa la sumatoria de los puntajes, cuenta el total de valoraciones y actualiza la entidad del vendedor utilizando la función modular de promedio.

Utilidad para el modelo: Automatiza y estandariza el proceso de consolidación de la reputación comercial dentro del sistema. Garantiza que el perfil de los vendedores refleje su desempeño real, actualizando de forma asíncrona sus métricas en base al comportamiento individual de sus publicaciones evaluadas.

```

1 CREATE PROCEDURE calificar_vendedor(
2     IN p_id_vendedor INT
3 )
4 BEGIN
5     DECLARE v_existe_vendedor INT DEFAULT 0;
6     DECLARE v_calificacion DECIMAL(2,1);
7     DECLARE v_suma DECIMAL(20,2) DEFAULT 0.0;
8     DECLARE v_cantidad BIGINT DEFAULT 0;
9     DECLARE v_promedio DECIMAL(2,1);
10    DECLARE v_fin BOOLEAN DEFAULT FALSE;
11
12    DECLARE cursor_calificaciones CURSOR FOR
13        SELECT producto.calificacion
14        FROM PUBLICACION AS publicacion
15        INNER JOIN PRODUCTO AS producto
16            ON producto.id_publicacion = publicacion.id_publicacion
17        WHERE publicacion.id_vendedor = p_id_vendedor
18        AND producto.calificacion IS NOT NULL;
19
20    DECLARE CONTINUE HANDLER FOR NOT FOUND
21        SET v_fin = TRUE;
22
23    SELECT COUNT(*)
24    INTO v_existe_vendedor
25    FROM VENDEDOR
26    WHERE id_vendedor = p_id_vendedor;
27
28    IF v_existe_vendedor = 0 THEN
29        SIGNAL SQLSTATE '45000'
30        SET MESSAGE_TEXT = 'El vendedor indicado no existe';
31    END IF;
32
33    OPEN cursor_calificaciones;
34
35    recorrer_calificaciones: LOOP
36        FETCH cursor_calificaciones INTO v_calificacion;
37
38        IF v_fin THEN
39            LEAVE recorrer_calificaciones;
40        END IF;
41
42        SET v_suma = v_suma + v_calificacion;
43        SET v_cantidad = v_cantidad + 1;
44    END LOOP;
45
46    CLOSE cursor_calificaciones;
47

```

```

48 SET v_promedio = promedio(v_suma, v_cantidad);
49
50 UPDATE VENDEDOR
51 SET calificacion = v_promedio
52 WHERE id_vendedor = p_id_vendedor;
53 END

```

4.3. Funciones

Las funciones se implementaron para abstraer cálculos matemáticos y lógicos repetitivos (como promedios estadísticos o algoritmos de calificación y fiabilidad). Esto permite que sean invocadas de manera escalar directamente dentro de las consultas SELECT o procedimientos almacenados, promoviendo la modularidad y evitando la duplicación de código.

4.3.1 Promedio

Descripción: Función determinística que recibe como entrada la sumatoria total y el conteo absoluto de elementos para retornar el promedio aritmético, redondeado a un solo decimal. Incluye una validación condicional estricta para evitar errores críticos de división por cero.

Utilidad para el modelo: Proporciona un bloque fundamental de construcción reutilizable en todo el modelo de datos. Asegura la consistencia general en el redondeo y permite que otras estructuras complejas deleguen el cálculo de medias sin preocuparse por manejar escenarios nulos o matemáticamente inválidos.

```

1 CREATE FUNCTION promedio(
2   p_total DECIMAL(20,2),
3   p_cantidad BIGINT
4 )
5 RETURNS DECIMAL(2,1)
6 DETERMINISTIC
7 BEGIN
8   IF p_cantidad = 0 THEN
9     RETURN 0.0;
10  END IF;
11
12  RETURN ROUND(p_total / p_cantidad, 1);
13 END

```

4.3.2 Probabilidad Producto

Descripción: Esta función calcula un puntaje de fiabilidad estadística probabilística asociado a un producto en particular, suavizando la calificación obtenida tomando como referencia un factor constante y la calificación general del mercado (media bayesiana) para productos con historial corto.

Utilidad para el modelo: Permite al sistema establecer algoritmos de ordenamiento mucho más inteligentes para las búsquedas. Destaca a los productos que no solo tienen buena nota, sino que también poseen un historial sólido de ventas, mitigando el sesgo en artículos con muy pocas calificaciones pero con puntajes perfectos.

```

1 CREATE FUNCTION calcular_probabilidad_producto(p_id_producto INT)
2 RETURNS DECIMAL(10,6)
3 READS SQL DATA
4 BEGIN
5   DECLARE v_vt INT DEFAULT 0;
6   DECLARE v_c DECIMAL(10,4) DEFAULT 0.0000;
7   DECLARE v_cm DECIMAL(10,4) DEFAULT 0.0000;
8   DECLARE v_k DECIMAL(10,4) DEFAULT 10.0000;
9   DECLARE v_fiabilidad DECIMAL(10,4) DEFAULT 0.0000;

```

```

10 SELECT v.ventas_completadas, v.calificacion
11 INTO v_vt, v_c
12 FROM PRODUCTO p
13 INNER JOIN PUBLICACION pub ON p.id_publicacion = pub.id_publicacion
14 INNER JOIN VENDEDOR v ON pub.id_vendedor = v.id_vendedor
15 WHERE p.id_producto = p_id_producto;
16
17 SELECT AVG(calificacion)
18 INTO v_cm
19 FROM VENDEDOR;
20
21 IF (v_vt + v_k) > 0 THEN
22     SET v_fiabilidad =
23     ((v_vt / (v_vt + v_k)) * v_c) +
24     ((v_k / (v_vt + v_k)) * v_cm);
25 ELSE
26     SET v_fiabilidad = 0.0000;
27 END IF;

```

4.3.3 Cálculo de fiabilidad del vendedor

Descripción: Función homóloga a la del producto, pero orientada a otorgar un puntaje global para la entidad del vendedor mediante una media estadística (bayesiana). Interpola la propia calificación del ofertante con el promedio total de los vendedores en plataforma ponderado por su volumen histórico de ventas completadas.

Utilidad para el modelo: Actúa como la pieza clave del algoritmo del sistema de reputación oficial. Ofrece una métrica equitativa que recompensa a los perfiles veteranos con consistencia comercial comprobada, lo que resulta ideal para posicionarlos de forma confiable dentro de los rankings de visualización y vistas precalculadas.

```

1 CREATE FUNCTION calcular_fiabilidad_vendedor(p_id_vendedor INT)
2 RETURNS DECIMAL(10,4)
3 READS SQL DATA
4 BEGIN
5     DECLARE v_vt INT DEFAULT 0;
6     DECLARE v_c DECIMAL(10,4) DEFAULT 0.0000;
7     DECLARE v_cm DECIMAL(10,4) DEFAULT 0.0000;
8     DECLARE v_k DECIMAL(10,4) DEFAULT 10.0000;
9     DECLARE v_fiabilidad DECIMAL(10,4) DEFAULT 0.0000;
10
11     SELECT ventas_completadas, calificacion
12     INTO v_vt, v_c
13     FROM VENDEDOR
14     WHERE id_vendedor = p_id_vendedor;
15
16     SELECT AVG(calificacion)
17     INTO v_cm
18     FROM VENDEDOR;
19
20     IF (v_vt + v_k) > 0 THEN
21         SET v_fiabilidad =
22         ((v_vt / (v_vt + v_k)) * v_c) +
23         ((v_k / (v_vt + v_k)) * v_cm);
24     ELSE

```

```
25 SET v_fiabilidad = 0.0000;  
26 END IF;  
27  
28 RETURN v_fiabilidad;  
29 END //
```


Capítulo 5

Batería de Consultas SQL

A continuación, se presenta la batería de consultas requerida para la verificación analítica del modelo de datos de UniTrade. Se construyeron 15 consultas de complejidad progresiva clasificadas en seis niveles (N1 a N6) orientadas a resolver interrogantes clave del modelo de negocio, tomando en cuenta las actualizaciones arquitectónicas del esquema.

5.1. Nivel 1: Selección y Filtrado

5.1.1 Consulta N1-01

Descripción: Identifica directamente los identificadores de los productos que han sido asociados a una materia específica en nuestra base de datos relacional.

Utilidad para el modelo: Esta consulta base permite comprobar rápidamente que las inserciones en la tabla asociativa MATERIA_PRODUCTO se han llevado a cabo exitosamente, siendo un pilar fundamental para las búsquedas académicas del sistema.

```
1 -- Consulta N1-01: Identifica los productos asociados a una materia específica.
2 SELECT id_producto
3 FROM MATERIA_PRODUCTO
4 WHERE id_materia = 5;
```

#	id_producto
1	73
2	105
3	162
4	215
5	454
6	482
7	487

5.1.2 Consulta N1-02

Descripción: Retorna el detalle básico y el precio de productos pertenecientes a una categoría puntual, aplicando un filtro de rango económico y comprobando que exista disponibilidad.

Utilidad para el modelo: Demuestra la viabilidad de implementar búsquedas con filtros paramétricos dentro de la aplicación, permitiendo a los estudiantes encontrar materiales ajustados a su presupuesto, utilizando el nuevo esquema con CATEGORIA_PRODUCTO.

```

1  -- Consulta N1-02: Retorna los productos de una categoría específica dentro de un rango de precio.
2  SELECT cp.id_producto, p.precio, c.nombre
3  FROM PRODUCTO p
4  JOIN CATEGORIA_PRODUCTO cp ON p.id_producto = cp.id_producto
5  JOIN CATEGORIA c ON cp.id_categoria = c.id_categoria
6  WHERE p.precio BETWEEN 20000 AND 80000
7  AND c.nombre = 'Libros y Textos';

```

#	id_producto	precio	nombre
1	20	46788.00	Libros y Textos
2	35	66703.00	Libros y Textos
3	47	53928.00	Libros y Textos
4	55	43215.00	Libros y Textos
5	77	72418.00	Libros y Textos
6	87	65067.00	Libros y Textos
7	129	56760.00	Libros y Textos
8	130	58417.00	Libros y Textos
9	146	53302.00	Libros y Textos
10	171	76471.00	Libros y Textos
11	214	47018.00	Libros y Textos
12	227	22861.00	Libros y Textos
13	228	58723.00	Libros y Textos
14	269	46040.00	Libros y Textos
15	281	64869.00	Libros y Textos
16	285	32711.00	Libros y Textos
17	291	75138.00	Libros y Textos
18	293	58988.00	Libros y Textos
19	314	62009.00	Libros y Textos
20	323	55075.00	Libros y Textos
21	360	72121.00	Libros y Textos
22	399	68152.00	Libros y Textos
23	418	51365.00	Libros y Textos
24	423	41646.00	Libros y Textos
25	432	31544.00	Libros y Textos
26	440	53321.00	Libros y Textos
27	450	52866.00	Libros y Textos
28	461	28697.00	Libros y Textos
29	487	48585.00	Libros y Textos

5.1.3 Consulta N1-03

Descripción: Muestra la información de las publicaciones correspondientes a productos físicos que en el momento actual tienen un estado de publicación activo.

Utilidad para el modelo: Es el listado predeterminado que alimentará el feed o la pantalla de inicio (marketplace) visible para cualquier usuario que busque artículos físicos al ingresar.

```

1  -- Consulta N1-03: Muestra las publicaciones de productos físicos que se encuentran activas
2  actualmente.
3  SELECT id_publicacion, titulo, descripcion, fecha_publicacion
4  FROM PUBLICACION
5  WHERE estado_publicacion = 'Activa' AND tipo_item = 'Producto';

```

#	id_publicacion	titulo	descripcion	fecha_publicacion
1	15	Libro de Cálculo de Larson 11Ed (...)	NULL	2026-07-23 23:49:13
2	18	Escalímetro profesional (Ref: 38)	Implementos de dibujo técnico semi...	2026-07-23 23:49:13
3	29	Kit de Laboratorio de Química (Ref:...	Set completo de cristalería resisten...	2026-07-23 23:49:13
4	50	Calculadora Científica Casio (Ref: ...)	Calculadora con funciones matricial...	2026-07-23 23:49:13
5	52	Kit de Laboratorio de Química (Ref:...	Set completo de cristalería resisten...	2026-07-23 23:49:13
6	76	Kit de Laboratorio de Química (Ref:...	Set completo de cristalería resisten...	2026-07-23 23:49:13
7	85	Prototyping Breadboard (Ref: 63)	Placa de pruebas protoboard con j...	2026-07-23 23:49:13
8	119	Escalímetro profesional (Ref: 50)	Implementos de dibujo técnico semi...	2026-07-23 23:49:13
9	134	Libro de Cálculo de Larson 11Ed (...)	Libro en excelente estado, pasta ori...	2026-07-23 23:49:13
10	140	Estetoscopio Littmann Classic III (R...	Uso exclusivo de área de salud.	2026-07-23 23:49:13
11	144	Tarjeta Arduino Uno R3 (Ref: 16)	NULL	2026-07-23 23:49:13
12	148	Tarjeta Arduino Uno R3 (Ref: 64)	NULL	2026-07-23 23:49:13
13	150	Multímetro Digital UT33C+ (Ref: 23)	NULL	2026-07-23 23:49:13
14	159	Calculadora Científica Casio (Ref: ...)	Calculadora con funciones matricial...	2026-07-23 23:49:13
15	183	Libro de Bases de Datos (Ref: 52)	Edición en español, clave para inge...	2026-07-23 23:49:13
16	184	Calculadora Científica Casio (Ref: ...)	Calculadora con funciones matricial...	2026-07-23 23:49:13
17	201	Libro de Bases de Datos (Ref: 22)	Edición en español, clave para inge...	2026-07-23 23:49:13
18	241	Tarjeta Arduino Uno R3 (Ref: 14)	NULL	2026-07-23 23:49:13
19	264	Estetoscopio Littmann Classic III (R...	Uso exclusivo de área de salud.	2026-07-23 23:49:13
20	277	Kit de Laboratorio de Química (Ref:...	NULL	2026-07-23 23:49:13
21	304	Libro de Cálculo de Larson 11Ed (...)	Libro en excelente estado, pasta ori...	2026-07-23 23:49:13
22	308	Calculadora Científica Casio (Ref: ...)	NULL	2026-07-23 23:49:13
23	312	Escalímetro profesional (Ref: 92)	Implementos de dibujo técnico semi...	2026-07-23 23:49:13
24	315	Física Universitaria Sears (Ref: 36)	Conjunto de ambos volúmenes em...	2026-07-23 23:49:13
25	318	Kit de Laboratorio de Química (Ref:...	Set completo de cristalería resisten...	2026-07-23 23:49:13
26	323	Física Universitaria Sears (Ref: 13)	Conjunto de ambos volúmenes em...	2026-07-23 23:49:13
27	326	Libro de Bases de Datos (Ref: 67)	NULL	2026-07-23 23:49:13
28	331	Física Universitaria Sears (Ref: 15)	Conjunto de ambos volúmenes em...	2026-07-23 23:49:13
29	336	Física Universitaria Sears (Ref: 20)	Conjunto de ambos volúmenes em...	2026-07-23 23:49:13
30	351	Kit de Laboratorio de Química (Ref:...	Set completo de cristalería resisten...	2026-07-23 23:49:13
31	361	Libro de Cálculo de Larson 11Ed (...)	Libro en excelente estado, pasta ori...	2026-07-23 23:49:13

5.2. Nivel 2: JOINS Múltiples

5.2.1 Consulta N2-01

Descripción: Presenta el catálogo de materias de la universidad y enlaza a través de todas las jerarquías de producto hasta llegar a los administradores responsables de la moderación de las publicaciones de dichos artículos (incluyendo materias sin artículos ni moderadores).

Utilidad para el modelo: Permite tener un panorama global de la carga de moderación de artículos dividida por áreas de conocimiento o materias, utilizando un `LEFT JOIN` para no excluir materias que aún no cuenten con productos vinculados.

```

1  -- Consulta N2-01: Muestra los administradores asociados a cada materia, incluyendo materias sin
2  -- administradores.
3  SELECT m.nombre_materia, a.id_administrador, a.area_soporte
   FROM MATERIA m

```

```

4 LEFT JOIN MATERIA_PRODUCTO mp ON m.id_materia = mp.id_materia
5 LEFT JOIN PRODUCTO p ON mp.id_producto = p.id_producto
6 LEFT JOIN PUBLICACION pu ON p.id_publicacion = pu.id_publicacion
7 LEFT JOIN ADMINISTRADOR a ON pu.id_administrador_moderador = a.id_administrador;

```

#	nombre_materia	id_administrador	area_soporte
1	Bases de Datos I	6	Verificación de Identidad
2	Bases de Datos I	NULL	NULL
3	Bases de Datos I	2	Moderación de Contenido
4	Bases de Datos I	6	Verificación de Identidad
5	Bases de Datos I	NULL	NULL
6	Programación Orientada a Objetos	NULL	NULL
7	Programación Orientada a Objetos	5	Atención al Estudiante
8	Programación Orientada a Objetos	NULL	NULL
9	Programación Orientada a Objetos	4	Gestión de Préstamos y Multas
10	Programación Orientada a Objetos	3	Seguridad y Cuentas
11	Programación Orientada a Objetos	7	Resolución de Disputas
12	Programación Orientada a Objetos	5	Atención al Estudiante
13	Cálculo Diferencial	1	Soporte Técnico de Plataforma
14	Cálculo Diferencial	3	Seguridad y Cuentas
15	Cálculo Diferencial	4	Gestión de Préstamos y Multas
16	Física Mecánica	5	Atención al Estudiante
17	Física Mecánica	6	Verificación de Identidad
18	Física Mecánica	4	Gestión de Préstamos y Multas
19	Física Mecánica	7	Resolución de Disputas
20	Estructuras de Datos	8	Soporte de Pagos
21	Estructuras de Datos	3	Seguridad y Cuentas
22	Estructuras de Datos	NULL	NULL
23	Estructuras de Datos	10	Administración General
24	Estructuras de Datos	NULL	NULL
25	Estructuras de Datos	6	Verificación de Identidad
26	Estructuras de Datos	6	Verificación de Identidad
27	Arquitectura de Software	2	Moderación de Contenido
28	Arquitectura de Software	3	Seguridad y Cuentas
29	Arquitectura de Software	10	Administración General
30	Arquitectura de Software	6	Verificación de Identidad
31	Arquitectura de Software	3	Seguridad y Cuentas

5.2.2 Consulta N2-02

Descripción: Identifica qué vendedor y mediante qué título de publicación está ofreciendo artículos que son pertinentes para una asignatura en específico (en este ejemplo, la asignatura con ID 1).

Utilidad para el modelo: Resulta vital para modelar el sistema de recomendaciones; si el sistema sabe que el estudiante cursa "Bases de Datos I", puede sugerirle directamente a los vendedores que ofrecen productos con esta etiqueta.

```

1  -- Consulta N2-02: Identifica los productos ofrecidos por vendedores para una materia específica.
2  SELECT v.id_vendedor, u.nombre_completo, pu.titulo AS producto_titulo, m.nombre_materia
3  FROM VENDEDOR v
4  JOIN USUARIO u ON v.id_vendedor = u.id_usuario
5  JOIN PUBLICACION pu ON v.id_vendedor = pu.id_vendedor
6  JOIN PRODUCTO p ON pu.id_publicacion = p.id_publicacion
7  JOIN MATERIA_PRODUCTO mp ON p.id_producto = mp.id_producto
8  JOIN MATERIA m ON mp.id_materia = m.id_materia
9  WHERE m.id_materia = 1;

```

#	id_vendedor	nombre_completo	producto_titulo	nombre_materia
1	100	Aldemar Esteban Salgado	Estetoscopio Littmann Classic III (R...	Bases de Datos I
2	93	Bernardo Sierra	Calculadora Científica Casio (Ref: ...	Bases de Datos I
3	204	Azucena Díaz García	Multímetro Digital UT33C+ (Ref: 44)	Bases de Datos I
4	119	Blanca Isabel Moreno	Tarjeta Arduino Uno R3 (Ref: 63)	Bases de Datos I
5	198	Luz Silva Mosquera	Multímetro Digital UT33C+ (Ref: 81)	Bases de Datos I

5.2.3 Consulta N2-03

Descripción: Vincula los servicios ofertados con el nombre del responsable de su verificación (administrador) y la reputación del vendedor que brinda dicho servicio académico.

Utilidad para el modelo: Provee una herramienta de auditoría de calidad de servicios; si un servicio con alta demanda tiene un vendedor con mala calificación, el administrador moderador enlazado puede intervenir rápidamente.

```

1  -- Consulta N2-03: Vincula los servicios ofrecidos con el nombre del administrador moderador y la
2  calificación del vendedor.
3  SELECT pu.titulo, s.modalidad, v.calificacion AS rating_vendedor, a.area_soporte
4  FROM PUBLICACION pu
5  JOIN SERVICIO s ON pu.id_publicacion = s.id_publicacion
6  JOIN VENDEDOR v ON pu.id_vendedor = v.id_vendedor
7  JOIN ADMINISTRADOR a ON pu.id_administrador_moderador = a.id_administrador;

```

#	titulo	modalidad	rating_vendedor	area_soporte
1	Corrección de Artículos	Presencial	4.0	Soporte Técnico de Plataforma
2	Clases de Física Mecánica	Virtual	10.0	Soporte Técnico de Plataforma
3	Corrección de Artículos	Presencial	10.0	Soporte Técnico de Plataforma
4	Corrección de Artículos	Presencial	0.0	Soporte Técnico de Plataforma
5	Corrección de Artículos	Virtual	10.0	Soporte Técnico de Plataforma
6	Asesoría en Programación	Presencial	3.7	Soporte Técnico de Plataforma
7	Corrección de Artículos	Presencial	3.6	Soporte Técnico de Plataforma
8	Tutoría de Cálculo Diferencial	Presencial	10.0	Soporte Técnico de Plataforma
9	Tutoría de Cálculo Diferencial	Virtual	4.9	Soporte Técnico de Plataforma
10	Clases de Física Mecánica	Virtual	4.5	Soporte Técnico de Plataforma
11	Clases de Física Mecánica	Virtual	3.8	Soporte Técnico de Plataforma
12	Corrección de Artículos	Presencial	4.0	Soporte Técnico de Plataforma
13	Clases de Física Mecánica	Presencial	0.0	Soporte Técnico de Plataforma
14	Corrección de Artículos	Presencial	4.8	Soporte Técnico de Plataforma
15	Tutoría de Cálculo Diferencial	Virtual	10.0	Soporte Técnico de Plataforma
16	Clases de Física Mecánica	Virtual	10.0	Soporte Técnico de Plataforma
17	Corrección de Artículos	Presencial	10.0	Soporte Técnico de Plataforma
18	Asesoría en Programación	Presencial	10.0	Soporte Técnico de Plataforma
19	Corrección de Artículos	Presencial	0.0	Soporte Técnico de Plataforma
20	Asesoría en Programación	Virtual	4.9	Soporte Técnico de Plataforma
21	Clases de Física Mecánica	Presencial	10.0	Soporte Técnico de Plataforma
22	Corrección de Artículos	Presencial	10.0	Soporte Técnico de Plataforma
23	Tutoría de Cálculo Diferencial	Presencial	0.0	Soporte Técnico de Plataforma
24	Clases de Física Mecánica	Virtual	10.0	Soporte Técnico de Plataforma
25	Clases de Física Mecánica	Presencial	3.9	Soporte Técnico de Plataforma
26	Tutoría de Cálculo Diferencial	Virtual	10.0	Soporte Técnico de Plataforma
27	Clases de Física Mecánica	Presencial	10.0	Soporte Técnico de Plataforma
28	Corrección de Artículos	Virtual	0.0	Soporte Técnico de Plataforma
29	Clases de Física Mecánica	Presencial	10.0	Soporte Técnico de Plataforma
30	Corrección de Artículos	Presencial	4.2	Soporte Técnico de Plataforma
31	Corrección de Artículos	Virtual	4.1	Soporte Técnico de Plataforma

5.2.4 Consulta N2-04

Descripción: Utiliza un LEFT JOIN entre la entidad base de usuarios y la entidad de vendedores para filtrar aquellos registros cuya correspondencia en la tabla de vendedores es nula, identificando a los usuarios que solo interactúan como compradores o usuarios pasivos.

Utilidad para el modelo: Permite conocer la tasa de conversión de usuarios a vendedores dentro de la plataforma, facilitando el análisis del comportamiento del consumidor y la planificación de estrategias para incentivar la publicación de productos.

```
1  -- Consulta N2-04: Filtra los usuarios registrados que nunca han actuado como vendedores.
2  SELECT us.nombre_completo, us.correo_estudiantil
3  FROM USUARIO us
4  LEFT JOIN VENDEDOR v ON us.id_usuario = v.id_vendedor
5  WHERE v.id_vendedor IS NULL;
```


#	nombre_completo	correo_estudiantil
1	Carmen Francia Bernal ...	dcortes129@uniandes.edu.co
2	Luz Valencia	libia42522@javeriana.edu.co
3	Alexander Alejandro Dí...	fperez795@unisabana.edu.co
4	Patricia Muñoz	rubysanchez833@udea.edu.co
5	Sandra Catherine Castro	oortiz600@javeriana.edu.co
6	Javier Johan Ortiz Tamayo	gloriamosquera555@udea.edu.co
7	Fernando Eduardo Zapata	svasquez309@uninorte.edu.co
8	Yaneth Nubia Giraldo Toro	eduardo49143@uis.edu.co
9	Juan Víctor Cardona Va...	salasesteban588@uninorte.edu.co
10	Sonia Barragán	carlosramirez949@univalle.edu.co
11	Jhon Ever Jiménez Her...	camila53399@udea.edu.co
12	Manuel Cuéllar Arango	camilomartinez149@javeriana.e...
13	José Martínez Galindo	perdomocamila189@uniandes.e...
14	Elena Pulido	davidhernandez683@unal.edu.co
15	Fernando Alexander He...	bvargas499@javeriana.edu.co
16	Rocío Ángela Lozano T...	alejandravega118@univalle.edu.co
17	Inés Zapata	torresarley465@unal.edu.co
18	Óscar Muñoz	patriciadaza881@unal.edu.co
19	Jhoana Rodríguez	jose08565@univalle.edu.co
20	Hernán Gómez Rivera	alexanderhernandez717@unino...
21	Gerardo Jean Ramírez ...	ortizluz411@uis.edu.co
22	María Ospina	galvismartin114@urosario.edu.co
23	Yina Londoño Estrada	alexandragallego956@udea.edu...
24	Johan García	ublanco912@uis.edu.co
25	Fabio Puentes	carlosgil310@unal.edu.co
26	Mauricio José Fernández	luz16687@udea.edu.co
27	Yuly Diana Muñoz	³² mairamoreno597@unisabana.e...
28	Amelia Ángela Morales	carmen44610@udea.edu.co

5.2.5 Consulta N2-05

Descripción: Busca coincidencias donde un individuo participa simultáneamente como el proveedor vendedor y como el usuario comprador del mismo tipo de bien físico.

Utilidad para el modelo: Funciona como un componente esencial de detección de fraudes (ej. autopublicaciones que buscan inflar de manera artificial las métricas de calificación o el total de ventas del usuario, así como revendedores que provocan inflación artificial en el precio de los productos).

```

1  -- Consulta N2-05: Busca usuarios que compren y vendan el mismo producto físico.
2  SELECT DISTINCT u.id_usuario, u.nombre_completo, p.id_producto, pu_compra.titulo
3  FROM USUARIO u
4  JOIN COMPRADOR c ON u.id_usuario = c.id_comprador
5  JOIN COMPRA co ON c.id_comprador = co.id_comprador
6  JOIN PUBLICACION pu_compra ON co.id_publicacion = pu_compra.id_publicacion
7  JOIN PRODUCTO p ON pu_compra.id_publicacion = p.id_publicacion
8  JOIN PUBLICACION pu_venta ON p.id_publicacion = pu_venta.id_publicacion
9  JOIN VENDEDOR v ON pu_venta.id_vendedor = v.id_vendedor
10 WHERE v.id_vendedor = u.id_usuario;

```

	id_usuario	nombre_completo	id_producto	titulo
▶	3	Antonio López Ruiz	464	Escalímetro profesional (Ref: 66)
	183	Ernesto Padilla	17	Calculadora Científica Casio (Ref: 74)

5.3. Nivel 3: Agregación

5.3.1 Consulta N3-01

Descripción: Realiza una sumatoria sobre el atributo `monto_total` de todas las compras finalizadas de cada vendedor, limitando la visualización a aquellos que sobrepasan un umbral mínimo de ingresos (50,000 pesos).

Utilidad para el modelo: Esta métrica posibilita determinar los "Top Sellers" de la plataforma.

```

1  -- Consulta N3-01: Calcula el ingreso total generado por cada vendedor en ventas finalizadas
2  -- superiores a un umbral.
3  SELECT pu.id_vendedor, u.nombre_completo, SUM(co.monto_total) AS ingreso_total
4  FROM COMPRA co
5  JOIN PUBLICACION pu ON co.id_publicacion = pu.id_publicacion
6  JOIN USUARIO u ON pu.id_vendedor = u.id_usuario
7  GROUP BY pu.id_vendedor, u.nombre_completo
8  HAVING SUM(co.monto_total) > 50000;

```

	id_vendedor	nombre_completo	ingreso_total
►	244	Jorge Navarro Gómez	1583533.21875
	123	Wilson Andrés Vargas	1776018.08984375
	252	Sofía Beatriz Salazar	2280699.375
	239	Yuri Paz	2083761.0234375
	18	María López Hernández	428054.5234375
	271	Rafael Vergara	1420592.7421875
	106	Eduardo Pérez González	1599057.814453125
	150	Andrés Pedro Espinosa Lozano	2018201.34765625
	60	Erika Linda Aguirre Contreras	1758078.625
	279	Paula Eugenia Villamizar Agudelo	2727613.6640625
	109	Isabel Camila Vera Giraldo	1932389.203125
	198	Luz Silva Mosquera	1804156.0234375
	128	Mauricio Restrepo Niño	2564352.84765625
	104	Herney Andrey Rojas López	1344925.7734375
	48	Marlén Guevara Castaño	1288141.09375
	2	Josefina Torres Flórez	1076663.671875
	152	Yaneth Pérez	1127185.080078125
	37	Jonathan Quintero	1493153.515625
	81	Antonio Didier Bermúdez Jiménez	974739.9609375
	146	Stella Laura Montenegro	892990.435546875
	30	Mauricio Gómez Pérez	1721046.513671875
	281	Dayana Sandra Parra	1616639.177734375
	101	María Rosa Ríos	416587.24609375
	247	Lucero Elena Monsalve	2500643.9921875

5.3.2 Consulta N3-02

Descripción: Aplica la función de agregación **MAX** agrupando por cada publicación registrada en la tabla de ofertas para calcular el valor monetario máximo que un ofertante ha estado dispuesto a pagar.

Utilidad para el modelo: Ayuda a evaluar la demanda y disposición de pago por producto en el sistema, lo que permite a los vendedores ajustar sus precios de mercado y a la plataforma identificar artículos con alta valoración.

```

1  -- Consulta N3-02: Calcula el mayor precio ofertado para cada publicación.
2  SELECT id_publicacion, MAX(monto_ofertado) AS mayor_precio_ofertado

```

```
3 FROM OFERTA
4 GROUP BY id_publicacion;
```

	id_publicacion	mayor_precio_ofertado
▶	22	114227
	25	97144.4
	26	68931.7
	29	81667
	32	61319.2
	33	102753
	36	92337.3
	54	74192.9
	58	44760.7
	65	16668.7
	66	46918.8
	73	30162.7
	78	33488.1
	80	113762
	84	81169
	86	119810
	98	114744
	99	25383.4
	101	25747.8
	106	54918.2
	108	55361.1
	111	110147
	116	108531
	126	32490.5

5.3.3 Consulta N3-03

Descripción: Cuantifica cuántas solicitudes de préstamo o alquiler de material académico ha realizado un comprador, excluyendo de la muestra a aquellos usuarios esporádicos mediante la cláusula **HAVING**.

Utilidad para el modelo: Permite construir métricas de fidelización y demanda de los préstamos, posibilitando promociones o penalidades dirigidas.

```

1  -- Consulta N3-03: Cuantifica la cantidad de préstamos solicitados por cada comprador, excluyendo
   esporádicos.
2  SELECT pr.id_comprador, u.nombre_completo, COUNT(pr.id_prestamo) AS total_prestamos
3  FROM PRESTAMO pr
4  JOIN USUARIO u ON pr.id_comprador = u.id_usuario
5  GROUP BY pr.id_comprador, u.nombre_completo
6  HAVING COUNT(pr.id_prestamo) > 2;

```

	metodo_pago	volumen_transaccional	cantidad_operaciones
▶	Efectivo	54720342.0546875	281
	Nequi	49212293.791015625	253
	Transferencia Bancaria	46291777.93359375	242
	Daviplata	49953995.310546875	274

5.3.4 Consulta N3-04

Descripción: Calcula la suma del monto transaccionado y el número total de operaciones agrupadas por método de pago, filtrando las compras realizadas durante los últimos seis meses mediante la función **DATE_SUB**.

Utilidad para el modelo: Proporciona métricas financieras periódicas clave sobre el flujo de dinero en la plataforma, permitiendo priorizar integraciones o alianzas estratégicas con las pasarelas de pago más utilizadas por los usuarios.

```

1  -- Consulta N3-04: Calcula el volumen transaccional y cantidad de operaciones por método de pago en
   el último semestre.
2  SELECT metodo_pago, SUM(monto_total) AS volumen_transaccional, COUNT(id_compra) AS
   cantidad_operaciones
3  FROM COMPRA
4  WHERE fecha_transaccion >= DATE_SUB(NOW(), INTERVAL 6 MONTH)
5  GROUP BY metodo_pago;

```

	id_comprador	nombre_completo	total_prestamos
▶	4	Carmen Francia Bernal Páez	4
	56	Mauricio José Fernández	4
	67	Andrés Jorge Parra	3
	69	Fernando Hernández	3
	90	Emilio Velasco	3
	121	Irene Martínez	3
	157	Camilo Gutiérrez Vargas	3
	218	Miguel Fajardo Romero	3
	236	William Luis Polo	3
	272	Dario Cristian Cortés	3
	282	Carlos Alberto Díaz Méndez	3
	285	Alejandra Yeimy Jaramillo	3
	299	Inés Estefanía Jaramillo Rin...	3

5.4. Nivel 4: Subconsultas

5.4.1 Consulta N4-01

Descripción: Identifica y enlista los productos registrados en el catálogo de la plataforma mediante una subconsulta con NOT EXISTS que verifica la ausencia de publicaciones de venta en estado activo asociadas a dicho producto.

Utilidad para el modelo: Permite detectar deficiencias o huecos en la oferta comercial disponible dentro del sistema, sirviendo como insumo para diseñar campañas de incentivo o notificar a los vendedores sobre oportunidades de mercado no atendidas.

```

1  -- Consulta N4-01: Enlista los productos que actualmente no tienen publicaciones de venta activas.
2  SELECT p.id_producto, pu.titulo
3  FROM PRODUCTO p INNER JOIN PUBLICACION pu ON p.id_publicacion = pu.id_publicacion
4  WHERE NOT EXISTS (
5      SELECT 1
6      FROM PUBLICACION v
7      WHERE v.id_publicacion = p.id_publicacion
8      AND v.estado_publicacion = 'Activa'
9  );

```

	id_producto	titulo
▶	1	Prototyping Breadboard (Ref: 63)
	2	Libro de Cálculo de Larson 11Ed (Ref: 50)
	3	Kit de Laboratorio de Química (Ref: 89)
	4	Tarjeta Arduino Uno R3 (Ref: 87)
	6	Prototyping Breadboard (Ref: 57)
	7	Estetoscopio Littmann Classic III (Ref: 89)
	9	Escalímetro profesional (Ref: 57)
	11	Física Universitaria Sears (Ref: 45)
	12	Estetoscopio Littmann Classic III (Ref: 10)
	13	Libro de Bases de Datos (Ref: 40)
	14	Estetoscopio Littmann Classic III (Ref: 15)
	15	Prototyping Breadboard (Ref: 52)
	16	Tarjeta Arduino Uno R3 (Ref: 71)
	18	Tarjeta Arduino Uno R3 (Ref: 36)
	20	Multímetro Digital UT33C+ (Ref: 63)
	21	Calculadora Científica Casio (Ref: 58)
	22	Estetoscopio Littmann Classic III (Ref: 51)
	23	Multímetro Digital UT33C+ (Ref: 40)
	24	Libro de Cálculo de Larson 11Ed (Ref: 42)
	25	Libro de Cálculo de Larson 11Ed (Ref: 77)
	26	Kit de Laboratorio de Química (Ref: 60)
	27	Calculadora Científica Casio (Ref: 33)
	28	Tarjeta Arduino Uno R3 (Ref: 86)
	29	Estetoscopio Littmann Classic III (Ref: 22)
	31	Calculadora Científica Casio (Ref: 85)

5.4.2 Consulta N4-02

Descripción: Obtiene el comprador que ha registrado el mayor gasto acumulado en la plataforma agrupando los montos totales de compra y comparándolos contra el valor máximo absoluto mediante el uso de subconsultas derivadas e instruidas en la cláusula **WHERE**.

Utilidad para el modelo: Permite identificar de manera precisa al cliente de mayor valor (*Top Spender* o usuario VIP) en la plataforma sin depender de cláusulas de ordenamiento directo con límites, facilitando la creación de programas de fidelización o beneficios exclusivos.

1 -- Consulta N4-02: Obtiene el comprador con el mayor gasto acumulado en la plataforma.

```

2 SELECT id_comprador, dinero_gastado
3 FROM (
4     SELECT id_comprador, SUM(monto_total) AS dinero_gastado
5     FROM COMPRA
6     GROUP BY id_comprador
7 ) AS totales
8 WHERE dinero_gastado = (
9     SELECT MAX(dinero_gastado)
10    FROM (
11         SELECT SUM(monto_total) AS dinero_gastado
12        FROM COMPRA
13        GROUP BY id_comprador
14     ) AS maximos
15 );

```

	id_comprador	dinero_gastado
▶	142	2085764.171875

5.4.3 Consulta N4-03

Descripción: Utiliza una división relacional mediante agrupamiento y filtrado de agregaciones para obtener los vendedores cuyo catálogo de productos asociados abarca la totalidad de las materias registradas, calculada dinámicamente mediante una subconsulta escalar.

Utilidad para el modelo: Permite destacar a los comerciantes más diversificados y completos de la red, facilitando la creación de programas de incentivos o insignias de reputación dentro del ecosistema de la plataforma.

```

1 -- Consulta N4-03: Obtiene los vendedores cuyo catálogo de productos abarca la totalidad de las
2   materias registradas.
3 SELECT pu.id_vendedor
4 FROM PUBLICACION pu
5 JOIN PRODUCTO pr ON pu.id_publicacion = pr.id_publicacion
6 JOIN MATERIA_PRODUCTO mp ON pr.id_producto = mp.id_producto
7 GROUP BY pu.id_vendedor
8 HAVING COUNT(DISTINCT mp.id_materia) = (SELECT COUNT(*) FROM MATERIA);

```

id_vendedor

Por el cuantificador universal implicado en el enunciado de esta consulta, lo normal será que la tabla resultante tenga pocos elementos; partiendo únicamente de las inserciones de prueba efectuadas por `02_dml.sql` -como se muestra en el pantallazo- no hay un vendedor en la base de datos que cubra todo el catálogo de materias.

5.4.4 Consulta N4-04

Descripción: Obtiene el listado de usuarios registrados como vendedores que en el momento no poseen ninguna oferta en estado activo, haciendo uso de una expresión de tabla común (CTE) y una subconsulta con la cláusula `NOT EXISTS`.

Utilidad para el modelo: Aporta métricas fundamentales sobre el nivel de inactividad de los vendedores registrados, siendo clave para activar campañas de reinteracción (*re-engagement*) y auditorías sobre cuentas inactivas.


```

1  -- Consulta N4-04: Lista los vendedores que actualmente no tienen ninguna publicación activa.
2  WITH VendedoresPublicadores AS (
3      SELECT id_vendedor FROM PUBLICACION WHERE estado_publicacion = 'Activa'
4  )
5  SELECT u.nombre_completo
6  FROM USUARIO u JOIN VENDEDOR v ON u.id_usuario = v.id_vendedor
7  WHERE NOT EXISTS (SELECT 1 FROM VendedoresPublicadores vp WHERE vp.id_vendedor = v.id_vendedor);

```

	nombre_completo
▶	Felipe Mateo Fonseca
	Andrés Pablo Arboleda Arango
	María López Hernández
	Luz María Rodríguez Vargas
	Bertha Angie González
	Eduardo Camilo Ramírez Herrera
	Erika Linda Aguirre Contreras
	Gloria Ortiz
	Manuel Jhon Salcedo
	Julián Cristobal Ocampo Guerrero
	Herney Andrey Rojas López
	Patricia Gutiérrez
	Marco Hernández Pérez
	Stella Laura Montenegro
	Luis Ramírez
	María Soto Pabón
	Andrea Aguirre Mosquera
	Candelaria Álvarez Montes
	Juan Felipe Navarro
	Diana Isabel Castro
	Miguel Fajardo Romero
	Marly Ramírez
	Sandra López Rivera
	William Luis Polo
	Dora Camarqo

5.4.5 Consulta N4-05

Descripción: Retorna una lista estructurada de las universidades y anexa una métrica volumétrica del total de usuarios inscritos pertenecientes a dicha alma mater mediante el uso de una subconsulta de tipo correlacionada dentro de la cláusula de selección principal.

Utilidad para el modelo: Otorga estadísticas de adopción de alto nivel cruciales para entender el nivel de tracción y expansión interinstitucional de UniTrade.

```
1  -- Consulta N4-05: Retorna las universidades y la cantidad total de usuarios registrados en cada
   una.
2  SELECT uni.nombre AS universidad,
3         (SELECT COUNT(*)
4          FROM USUARIO u
5          WHERE u.id_universidad = uni.id_universidad) AS total_usuarios
6  FROM UNIVERSIDAD uni;
```

	universidad	total_usuarios
►	Universidad Nacional de Colombia	28
	Universidad de los Andes	28
	Universidad Javeriana	32
	Universidad de Antioquia	37
	Universidad del Valle	25
	Universidad Industrial de Santander	26
	Universidad del Rosario	26
	Universidad de la Sabana	31
	Universidad EAFIT	29
	Universidad del Norte	38

5.5. Nivel 5: Funciones de Ventana

5.5.1 Consulta N5-01

Descripción: Utiliza la función analítica RANK() y la técnica de PARTITION BY para segmentar el inventario de artículos según su estado de conservación, y ordenar de mayor a menor precio para rankear los productos más costosos por segmento, sin perder el detalle del producto.

Utilidad para el modelo: Representa la herramienta primordial para que los usuarios premium localicen los artículos más valiosos y demandados en el apartado de bienes usados y bienes nuevos con absoluta agilidad analítica, permitiendo estudios de mercado de los productos.

```
1  -- Consulta N5-01: Genera un ranking de los productos más caros dentro de cada estado físico.
2  SELECT id_producto, estado_fisico, precio,
3         RANK() OVER (PARTITION BY estado_fisico ORDER BY precio DESC) AS ranking_precio
4  FROM PRODUCTO;
```

	id_producto	estado_fisico	precio	ranking_precio
▶	219	NUEVO	359878.00	1
	95	NUEVO	359624.00	2
	147	NUEVO	359588.00	3
	192	NUEVO	359460.00	4
	379	NUEVO	358825.00	5
	111	NUEVO	357347.00	6
	245	NUEVO	357178.00	7
	161	NUEVO	357012.00	8
	467	NUEVO	356738.00	9
	195	NUEVO	356617.00	10
	107	NUEVO	356412.00	11
	62	NUEVO	355736.00	12
	172	NUEVO	355400.00	13
	284	NUEVO	353031.00	14
	14	NUEVO	352686.00	15
	240	NUEVO	351779.00	16
	256	NUEVO	350778.00	17
	91	NUEVO	350724.00	18
	460	NUEVO	350664.00	19
	22	NUEVO	350274.00	20
	121	NUEVO	349522.00	21
	88	NUEVO	348827.00	22
	191	NUEVO	348292.00	23
	188	NUEVO	348009.00	24
	176	NUEVO	347219.00	25

5.5.2 Consulta N5-02

Descripción: Aplica una suma progresiva y temporal (SUM() OVER()) a cada individuo comprador, con el objetivo de exhibir el acumulado cronológico de su inversión total con cada nueva compra iterativa registrada.

Utilidad para el modelo: Sirve de base innegable para calcular la progresión del ticket de compra y

alimentar dashboards financieros dinámicos o crear "Niveles de cuenta" basados en el dinero gastado en la plataforma a través del tiempo.

```

1  -- Consulta N5-02: Calcula el historial acumulado del monto de compras por cada comprador a lo
2  largo del tiempo.
3  SELECT id_comprador, fecha_transaccion, monto_total,
4         SUM(monto_total) OVER (
5             PARTITION BY id_comprador
6             ORDER BY fecha_transaccion
7         ) AS gasto_acumulado
8  FROM COMPRA;

```

	id_comprador	fecha_transaccion	monto_total	gasto_acumulado
▶	2	2026-07-23 23:41:09	105509	506100.7578125
	2	2026-07-23 23:41:09	153248	506100.7578125
	2	2026-07-23 23:41:09	155045	506100.7578125
	2	2026-07-23 23:41:09	92298.7	506100.7578125
	3	2026-07-23 23:41:09	111238	1932150.99609375
	3	2026-07-23 23:41:09	153433	1932150.99609375
	3	2026-07-23 23:41:09	145318	1932150.99609375
	3	2026-07-23 23:41:09	254146	1932150.99609375
	3	2026-07-23 23:41:09	328244	1932150.99609375
	3	2026-07-23 23:41:09	321677	1932150.99609375
	3	2026-07-23 23:41:09	201692	1932150.99609375
	3	2026-07-23 23:41:09	22056.7	1932150.99609375
	3	2026-07-23 23:41:09	179104	1932150.99609375
	3	2026-07-23 23:41:09	215242	1932150.99609375
	4	2026-07-23 23:41:09	163882	1704153.03515625
	4	2026-07-23 23:41:09	235265	1704153.03515625
	4	2026-07-23 23:41:09	251064	1704153.03515625
	4	2026-07-23 23:41:09	55499.4	1704153.03515625
	4	2026-07-23 23:41:09	236852	1704153.03515625
	4	2026-07-23 23:41:09	106554	1704153.03515625
	4	2026-07-23 23:41:09	331421	1704153.03515625
	4	2026-07-23 23:41:09	323616	1704153.03515625
	5	2026-07-23 23:41:09	141550	445320.171875
	5	2026-07-23 23:41:09	303771	445320.171875

5.5.3 Consulta N5-03

Descripción: Esta consulta aplica el Teorema de Chebyshev para analizar el comportamiento de las probabilidades de venta de los productos del catálogo e identificar aquellos cuyo desempeño se desvía significativamente de la tendencia general. Para ello, primero calcula la probabilidad individual de venta de cada producto mediante una función especializada. Posteriormente obtiene la media y la desviación estándar de todas las probabilidades y establece los límites de Chebyshev para $k = 2$, lo que garantiza que al menos el 75% de los datos deberían encontrarse dentro de dicho intervalo. Finalmente, clasifica cada producto como un *Éxito Atípico*, *Ventas Muy Bajas* o *Comportamiento Normal*, dependiendo de su posición respecto a estos límites estadísticos.

Utilidad para el modelo: Esta consulta permite detectar automáticamente productos con comportamientos excepcionales sin asumir que las probabilidades siguen una distribución normal. Los resultados son útiles para identificar publicaciones con un rendimiento extraordinario que podrían destacarse o analizarse como casos de éxito, así como productos con un desempeño muy inferior al esperado que requieran ajustes en precio, descripción o estrategia de comercialización. De esta manera, el modelo incorpora un criterio estadístico robusto para apoyar la toma de decisiones y el monitoreo del rendimiento del catálogo.

```

1 WITH TablaProbabilidades AS (
2
3     SELECT
4         id_producto,
5         id_publicacion,
6         precio,
7         calcular_probabilidad_producto(id_producto) AS proba
8     FROM PRODUCTO
9 ),
10 MetricasChebyshev AS (
11
12     SELECT
13         AVG(proba) AS media_mu,
14         STDDEV(proba) AS desviacion_sigma
15     FROM TablaProbabilidades
16 )
17
18 SELECT
19     t.id_producto,
20     t.proba,
21     ROUND(m.media_mu, 6) AS media_del_catologo,
22     ROUND(m.desviacion_sigma, 6) AS desviacion_del_catologo,
23     ROUND(m.media_mu + (2 * m.desviacion_sigma), 6) AS limite_superior_chebyshev,
24     CASE
25         WHEN t.proba > (m.media_mu + (2 * m.desviacion_sigma)) THEN 'Éxito Atípico (Fuera del
26             limite Chebyshev)'
27         WHEN t.proba < (m.media_mu - (2 * m.desviacion_sigma)) THEN 'Ventas Muy Bajas'
28         ELSE 'Comportamiento Normal (Dentro del 75 por ciento esperado)'
29     END AS diagnostico_chebyshev
30 FROM TablaProbabilidades t
31 CROSS JOIN MetricasChebyshev m
32 ORDER BY t.proba DESC;

```

5.6. Nivel 6: Vistas

5.6.1 Consulta N6-01

Descripción: Genera una vista que segmenta los servicios según su modalidad (Presencial o Virtual) y aplica la función de ventana `RANK() OVER (PARTITION BY ...)` para particionar y jerarquizar las tarifas por hora en cada categoría.

	_producto	proba	media_del_catalogo	desviacion_del_catalogo	limite_superior_chebyshev	diagnostico_chebyshev
9		0.000000	0.001903	0.001863	0.005629	Comportamiento Normal (Dentro del 75 p...
6		0.000000	0.001903	0.001863	0.005629	Comportamiento Normal (Dentro del 75 p...
7		0.000000	0.001903	0.001863	0.005629	Comportamiento Normal (Dentro del 75 p...
2		0.000000	0.001903	0.001863	0.005629	Comportamiento Normal (Dentro del 75 p...
3		0.000000	0.001903	0.001863	0.005629	Comportamiento Normal (Dentro del 75 p...
4		0.000000	0.001903	0.001863	0.005629	Comportamiento Normal (Dentro del 75 p...
5		0.000000	0.001903	0.001863	0.005629	Comportamiento Normal (Dentro del 75 p...
6		0.000000	0.001903	0.001863	0.005629	Comportamiento Normal (Dentro del 75 p...
7		0.000000	0.001903	0.001863	0.005629	Comportamiento Normal (Dentro del 75 p...
8		0.000000	0.001903	0.001863	0.005629	Comportamiento Normal (Dentro del 75 p...
9		0.000000	0.001903	0.001863	0.005629	Comportamiento Normal (Dentro del 75 p...
10		0.000000	0.001903	0.001863	0.005629	Comportamiento Normal (Dentro del 75 p...
11		0.000000	0.001903	0.001863	0.005629	Comportamiento Normal (Dentro del 75 p...
12		0.000000	0.001903	0.001863	0.005629	Comportamiento Normal (Dentro del 75 p...
13		0.000000	0.001903	0.001863	0.005629	Comportamiento Normal (Dentro del 75 p...
14		0.000000	0.001903	0.001863	0.005629	Comportamiento Normal (Dentro del 75 p...
15		0.000000	0.001903	0.001863	0.005629	Comportamiento Normal (Dentro del 75 p...
16		0.000000	0.001903	0.001863	0.005629	Comportamiento Normal (Dentro del 75 p...
17		0.000000	0.001903	0.001863	0.005629	Comportamiento Normal (Dentro del 75 p...
18		0.000000	0.001903	0.001863	0.005629	Comportamiento Normal (Dentro del 75 p...
19		0.000000	0.001903	0.001863	0.005629	Comportamiento Normal (Dentro del 75 p...
20		0.000000	0.001903	0.001863	0.005629	Comportamiento Normal (Dentro del 75 p...

Utilidad para el modelo: Permite realizar análisis comparativos de precios de mercado por segmento, facilitando la identificación de los servicios prémium dentro del catálogo de la plataforma.

```

1  -- Consulta N6-01: Crea una vista con el ranking de los servicios más caros divididos por su
2  -- modalidad.
3  CREATE OR REPLACE VIEW vw_ranking_servicios_modalidad AS
4  SELECT pu.titulo,
5  s.modalidad,
6  s.tarifa_por_hora,
7  RANK() OVER (PARTITION BY s.modalidad ORDER BY s.tarifa_por_hora DESC) AS ranking_precio
8  FROM SERVICIO s
9  JOIN PUBLICACION pu ON s.id_publicacion = pu.id_publicacion;

```

[illegible]

5.6.2 Consulta N6-02

Descripción: Define una vista analítica que utiliza una función de agregación sobre ventana `SUM()` `OVER (ORDER BY ... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)` para calcular el avance en el tiempo del flujo de caja acumulado.

Utilidad para el modelo: Es fundamental para el seguimiento financiero e indicadores ejecutivos (KPIs), proporcionando una trazabilidad clara del crecimiento monetario sin sobrecargar la base con cálculos manuales.

```
1  -- Consulta N6-02: Crea una vista del acumulado histórico de ingresos financieros diarios de la
   plataforma.
2  CREATE OR REPLACE VIEW vw_ingresos_acumulados AS
3  SELECT fecha_transaccion,
4         monto_total,
5         SUM(monto_total) OVER (
6         ORDER BY fecha_transaccion
7         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
8         ) AS ingreso_acumulado
9  FROM COMPRA;
```

	fecha_transaccion	monto_total	ingreso_acumulado
►	2026-07-24 19:30:40	61584.5	61584.4609375
	2026-07-24 19:30:40	60717.5	122301.921875
	2026-07-24 19:30:40	269644	391946.046875
	2026-07-24 19:30:40	77509.9	469455.8984375
	2026-07-24 19:30:40	81621.9	551077.828125
	2026-07-24 19:30:40	344769	895846.984375
	2026-07-24 19:30:40	63232.6	959079.59375
	2026-07-24 19:30:40	42934.5	1002014.08203125
	2026-07-24 19:30:40	111238	1113251.76953125
	2026-07-24 19:30:40	333363	1446614.89453125
	2026-07-24 19:30:40	110814	1557428.88671875
	2026-07-24 19:30:40	125759	1683187.52734375
	2026-07-24 19:30:40	46110.8	1729298.34765625
	2026-07-24 19:30:40	343010	2072308.44140625
	2026-07-24 19:30:40	74177.4	2146485.79296875
	2026-07-24 19:30:40	293252	2439738.19921875
	2026-07-24 19:30:40	314697	2754435.60546875
	2026-07-24 19:30:40	247850	3002285.24609375
	2026-07-24 19:30:40	240316	3242601.68359375
	2026-07-24 19:30:40	187825	3430426.87109375
	2026-07-24 19:30:40	133984	3564411.15234375
	2026-07-24 19:30:40	335673	3900084.49609375
	2026-07-24 19:30:40	165438	4065522.79296875
	2026-07-24 19:30:40	343005	4408527.57421875
	2026-07-24 19:30:40	29320.2	4437847.724609375
	2026-07-24 19:30:40	97584.1	4535431.787109375
	2026-07-24 19:30:40	226186	4761617.943359375
	2026-07-24 19:30:40	174612	4936230.333984375
	2026-07-24 19:30:40	166927	5103157.083984375
	2026-07-24 19:30:40	296845	5400002.021484375
	2026-07-24 19:30:40	41458.3	5441460.291015625
	2026-07-24 19:30:40	330413	5771873.666015625
	2026-07-24 19:30:40	308858	6080731.697265625
	2026-07-24 19:30:40	212506	6293237.712890625
	2026-07-24 19:30:40	45449.6	6338687.271484375

5.6.3 Consulta N6-03

Descripción: Crea una vista de supervisión para préstamos activos no devueltos en la fecha pactada y utiliza la función de ventana `RANK()` sobre los días de retraso para jerarquizar los casos de mayor severidad.

Utilidad para el modelo: Permite a los administradores priorizar la gestión de cobro o la emisión de sanciones sobre los deudores con los mayores niveles de mora en la entrega de ítems.

```

1  -- Consulta N6-03: Crea una vista de monitoreo administrativo para préstamos activos vencidos.
2  CREATE OR REPLACE VIEW vw_prestamos_riesgo AS
3  SELECT pr.id_prestamo,
4  uc.nombre_completo AS deudor,
5  pu.titulo AS item_prestado,
6  pr.fecha_devolucion_pactada,
7  DATEDIFF(NOW(), pr.fecha_devolucion_pactada) AS dias_retraso,
8  RANK() OVER (ORDER BY DATEDIFF(NOW(), pr.fecha_devolucion_pactada) DESC) AS ranking_mora
9  FROM PRESTAMO pr
10 JOIN COMPRADOR c ON pr.id_comprador = c.id_comprador
11 JOIN USUARIO uc ON c.id_comprador = uc.id_usuario
12 JOIN PUBLICACION pu ON pr.id_publicacion = pu.id_publicacion
13 WHERE pr.estado_prestamo = 'Activo'
14 AND pr.fecha_devolucion_pactada < NOW();

```

	id_prestamo	deudor	item_prestado	fecha_devolucion_pactada	dias_retraso	ranking_mora
--	-------------	--------	---------------	--------------------------	--------------	--------------

Lo ideal para el negocio es que esta vista permanezca como una tabla vacía, pues esto significa que no hay préstamos en mora y no es necesario aplicar sanciones.

5.6.4 Consulta N6-04

Descripción: Consolida métricas volumétricas de compras y ofertas por usuario en una vista e integra la función de ventana `DENSE_RANK()` para posicionar a los compradores con mayor nivel de actividad e interacción comercial.

Utilidad para el modelo: Otorga al equipo directivo una vista sintetizada del nivel de fidelidad e involucramiento de los usuarios en el mercado, simplificando la segmentación de la comunidad.

```

1  -- Consulta N6-04: Crea una vista de la actividad transaccional de los compradores, rankeados por
2  -- compras.
3  CREATE OR REPLACE VIEW vw_actividad_compradores AS
4  SELECT c.id_comprador,
5  u.nombre_completo,
6  (SELECT COUNT(*) FROM COMPRA co WHERE co.id_comprador = c.id_comprador) AS total_compras,
7  (SELECT COUNT(*) FROM OFERTA o WHERE o.id_comprador = c.id_comprador) AS total_ofertas,
8  DENSE_RANK() OVER (
9  ORDER BY (SELECT COUNT(*) FROM COMPRA co WHERE co.id_comprador = c.id_comprador) DESC
10 ) AS ranking_comprador
11 FROM COMPRADOR c
12 JOIN USUARIO u ON c.id_comprador = u.id_usuario;

```

	id_comprador	nombre_completo	total_compras	total_ofertas	ranking_comprador
▶	142	Marco Granados	11	0	1
	3	Antonio López Ruiz	10	0	2
	30	Mauricio Gómez Pérez	9	1	3
	40	Bertha Angie González	9	0	3
	71	Patricia Córdoba	9	0	3
	108	Marleny García	9	0	3
	242	Nora Gómez	9	1	3
	265	Manuel Restrepo Vega	9	0	3
	274	María Cecilia Camacho	9	0	3
	292	Martha Rodríguez	9	2	3
	4	Carmen Francia Bern...	8	0	4
	14	Astrid Giraldo Vásquez	8	1	4
	115	Deisy Bravo	8	2	4
	199	José Ayala Bernal	8	1	4
	221	Fernanda Ríos Morales	8	0	4
	280	Carlos José Narváez	8	0	4
	22	Juan Víctor Cardona ...	7	0	5
	47	Yina Londoño Estrada	7	3	5
	58	Amelia Ángela Morales	7	1	5
	93	Bernardo Sierra	7	3	5
	100	Aldemar Esteban Salg...	7	2	5
	112	Miguel Diego Gómez D...	7	0	5
	135	Carlos Luis Molina Qui...	7	0	5
	164	Andrés Edison Castillo	7	0	5
	167	Yair Moisés Sanabria ...	7	2	5
	218	Miguel Fajardo Romero	7	2	5
	224	Rubén Hugo Martínez...	7	0	5
	258	María Flor Gutiérrez G...	7	0	5
	270	Cecilia Rojas	7	1	5
	298	José Contreras	7	2	5
	25	Sonia Barragán	6	0	6
	28	José Martínez Galindo	6	0	6
	29	Gina Elvira Ayala Daza	6	0	6
	50	María Pinzón Galvis	6	3	6
	55	Yadira Bravo Buitrago	6	0	6

5.6.5 Consulta N6-05

Descripción: Esta vista genera un ranking de vendedores con base en una puntuación de fiabilidad calculada mediante una función especializada. Para garantizar la representatividad de los resultados, únicamente se consideran los vendedores que han completado al menos diez ventas. Además del ranking, la vista incorpora información como la posición secuencial, la calificación actual, el número de ventas completadas y la diferencia de puntuación respecto al vendedor inmediatamente superior, haciendo uso de funciones analíticas como `DENSE_RANK()`, `ROW_NUMBER()` y `LAG()`.

Utilidad para el modelo: Esta vista proporciona un mecanismo para evaluar y comparar el desem-

peño de los vendedores de manera objetiva, facilitando la identificación de los más confiables dentro de la plataforma. La información obtenida puede emplearse para mostrar clasificaciones a los usuarios, seleccionar vendedores destacados, establecer incentivos o realizar análisis sobre la evolución de la fiabilidad, contribuyendo así a mejorar la transparencia y la calidad del proceso de compraventa.

```

1  -- Consulta N6-05: Vista de ranking de vendedores según fiabilidad
2  CREATE OR REPLACE VIEW vista_ranking_vendedores AS
3  WITH VendedoresFiltrados AS (
4      SELECT
5          id_vendedor,
6          calificacion AS calificacion_actual,
7          ventas_completadas,
8          calcular_fiabilidad_vendedor(id_vendedor) AS puntuacion_fiabilidad
9      FROM VENDEDOR
10     WHERE ventas_completadas >= 10
11 ),
12 Ranking AS (
13     SELECT
14         id_vendedor,
15         calificacion_actual,
16         ventas_completadas,
17         puntuacion_fiabilidad,
18         ROUND(puntuacion_fiabilidad, 2) AS fiabilidad,
19
20         DENSE_RANK() OVER (
21             ORDER BY puntuacion_fiabilidad DESC
22         ) AS ranking_fiabilidad,
23
24         ROW_NUMBER() OVER (
25             ORDER BY puntuacion_fiabilidad DESC
26         ) AS posicion_fila,
27
28         LAG(puntuacion_fiabilidad) OVER (
29             ORDER BY puntuacion_fiabilidad DESC
30         ) AS fiabilidad_anterior
31
32     FROM VendedoresFiltrados
33 )
34 SELECT
35     id_vendedor,
36     calificacion_actual,
37     ventas_completadas,
38     fiabilidad,
39     ranking_fiabilidad,
40     posicion_fila,
41     ROUND(
42         COALESCE(
43             puntuacion_fiabilidad - fiabilidad_anterior,
44             0
45         ),
46         2
47     ) AS diferencia_con_puesto_anterior
48 FROM Ranking
49 ORDER BY ranking_fiabilidad;

```

	id_vendedor	calificacion_actual	ventas_completadas	fiabilidad	ranking_fiabilidad	posicion_fila	diferencia_con_puesto_anterior
▶	258	10.0	50	9.07	1	1	0.00
	126	10.0	48	9.03	2	2	-0.03
	183	10.0	48	9.03	2	3	0.00
	18	10.0	46	9.00	3	4	-0.04
	259	10.0	46	9.00	3	5	0.00
	277	10.0	46	9.00	3	6	0.00
	43	10.0	45	8.98	4	7	-0.02
	115	10.0	44	8.96	5	8	-0.02
	112	10.0	43	8.94	6	9	-0.02
	232	10.0	43	8.94	6	10	0.00
	252	10.0	43	8.94	6	11	0.00
	192	10.0	42	8.92	7	12	-0.02
	74	10.0	38	8.83	8	13	-0.09
	223	10.0	37	8.81	9	14	-0.03
	47	10.0	35	8.75	10	15	-0.05
	25	10.0	33	8.70	11	16	-0.06
	242	10.0	33	8.70	11	17	0.00
	3	10.0	32	8.67	12	18	-0.03
	288	10.0	31	8.63	13	19	-0.03
	228	10.0	30	8.60	14	20	-0.03

Figura 5.1: Resultado de la Consulta N5-03